Deep Learning Based AI Lab

Lecture: 26-07-2026



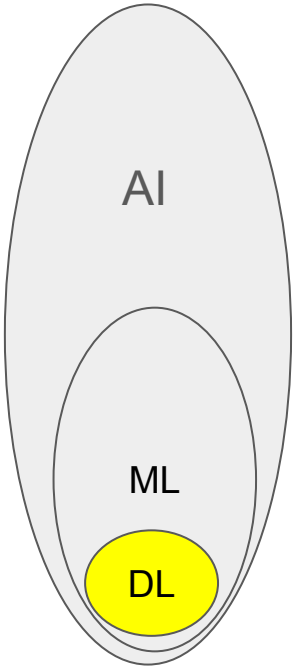
Sangeeta Biswas, Ph.D.
Associate Professor,
University of Rajshahi, Rajshahi-6205, Bangladesh

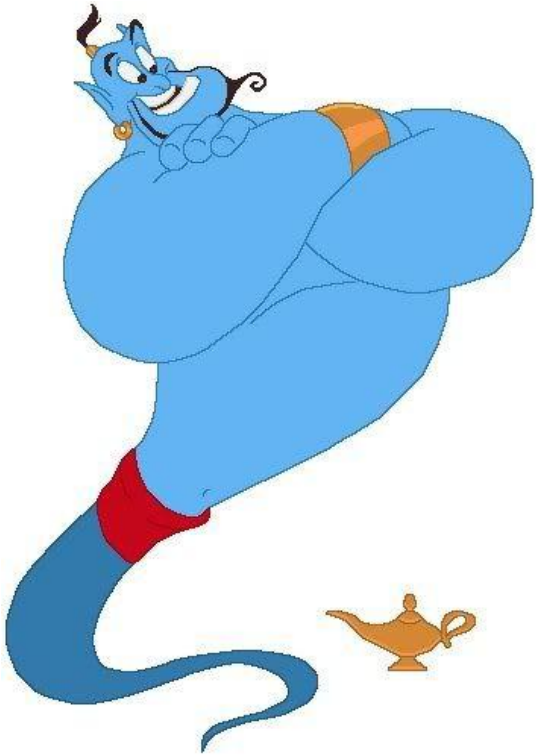
Introduction

Deep Learning based AI

Deep Learning (DL) based Artificial Intelligence (AI) achieved:

- Discriminative power
- Predictive Power
- Generative power
  - Because of the generative power, Deep Learning based AI could be a threat of human civilization if it is not handled in proper way.





Prerequisites of DL based AI Lab

- Python Programming
- Algebra
  - Variables, equations, Functions (linear & non-linear), Exponents, logarithms, Graphs of functions
- Linear Algebra
  - Vectors & matrices, Matrix multiplication, Vector spaces, Linear transformations
- Calculus
  - Derivatives (very important), Partial derivatives, Chain rule, Gradients, Optimization basics
- Probability & Statistics

What will We Use

- Linux Environment
- Tensorflow, Keras, PyTorch
- Python
- OpenCV
- Other Python Libraries such as matplotlib, scikit, pandas

Recommended Materials

Video Tutorials:

- Andrew Ng's Lectures
- MIT Introduction to Deep Learning
- Stanford Deep Learning

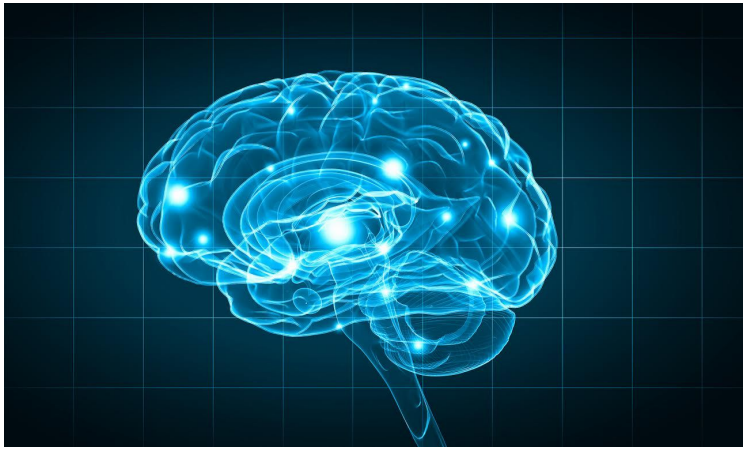
Reading Materials:

- Chollet's Book: Deep Learning with Python
- Andrew Ng's Materials:
- Goodfellow et al's book: Deep Learning
- Bishop's Book: Deep Learning Foundation and Concepts

History

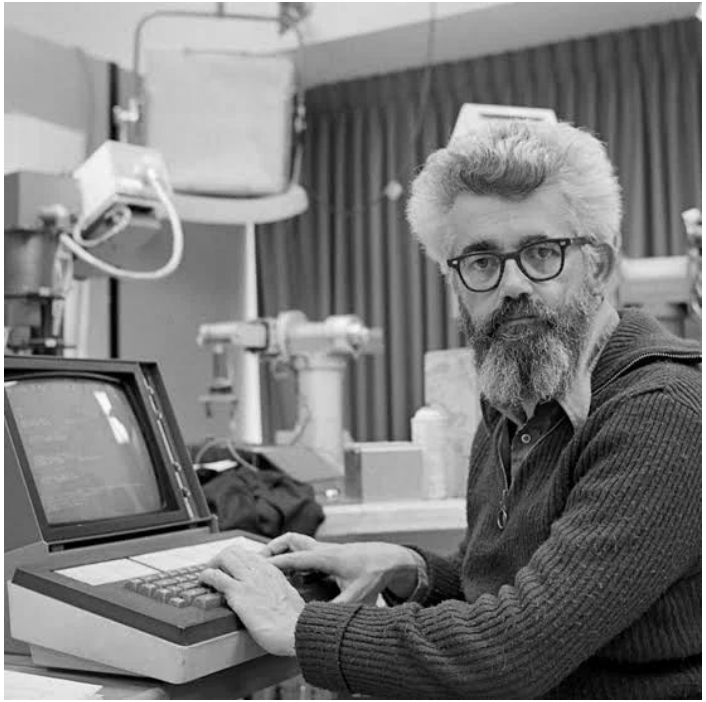
Intelligence

- Intelligence is the mental capacity for abstraction, logic, learning, planning, creativity, and problem-solving, allowing individuals to adapt to new environments and situations.
- It involves processing information, understanding complex ideas, and using experience to adaptively navigate life.
- AI is the ability of a system to **perform tasks that normally require human intelligence.**



Father of AI

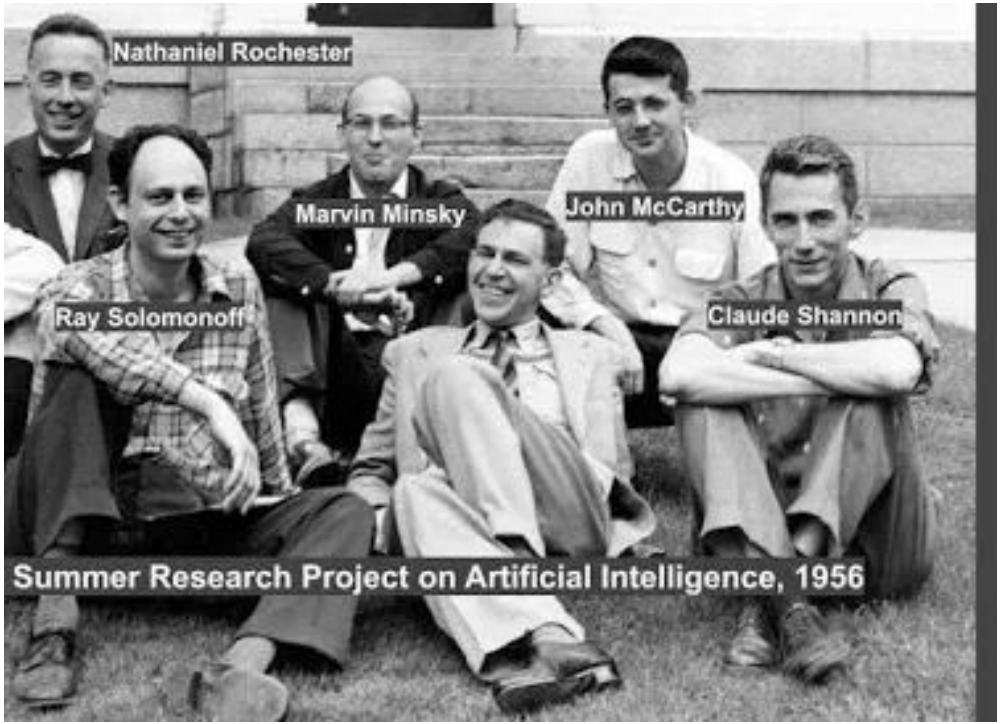
- **John McCarthy:** Known as the **“Father of AI”**
- Coined the term **“Artificial Intelligence”** in 1956
- Organized the famous **Dartmouth Conference** (birth of AI as a field)



Dartmouth Workshop in 1956

- This workshop is considered as the **Birthplace of AI** as a formal academic field.

- **Workshop Agenda:** Every aspect of learning or intelligence can be precisely described and simulated by a machine.



God Fathers of Artificial Neural Network (ANN)

- 2018 Turing Award winners for their foundational contributions to **Artificial Neural Networks (ANN)**.
- Yann LeCun: CNN
- Geoffrey Hinton: Backpropagation, Boltzmann Machines, and Deep Belief Networks
- Yoshua Bengio: RNN



Yann LeCun, Geoffrey Hinton, and Yoshua Bengio

2024 Nobel Prize in Physics

- Their work applies **principles from physics** to computational models.
- Developed networks capable of **storing, reconstructing, and recognizing patterns** in data.
- These contributions form a **core foundation of modern AI.**



John J. Hopfield and Geoffrey E. Hinton

Functions

Functions

- A function is a special rule or connection that links an input value to one and only one output value.
- Functions can be broadly classified into **two categories** based on the shape of their graphs and the relationship between input and output:
  - **Linear Functions**
    - Its graph is a **straight line**.
  - **Non-linear Functions**
    - Its graph is not a **straight line**.

ANN: A Forest of Functions

- An **Artificial Neural Network (ANN)** can be viewed as a **composition of multiple functions.**

  Suppose an ANN has *L* layers. The overall function learned by the network is

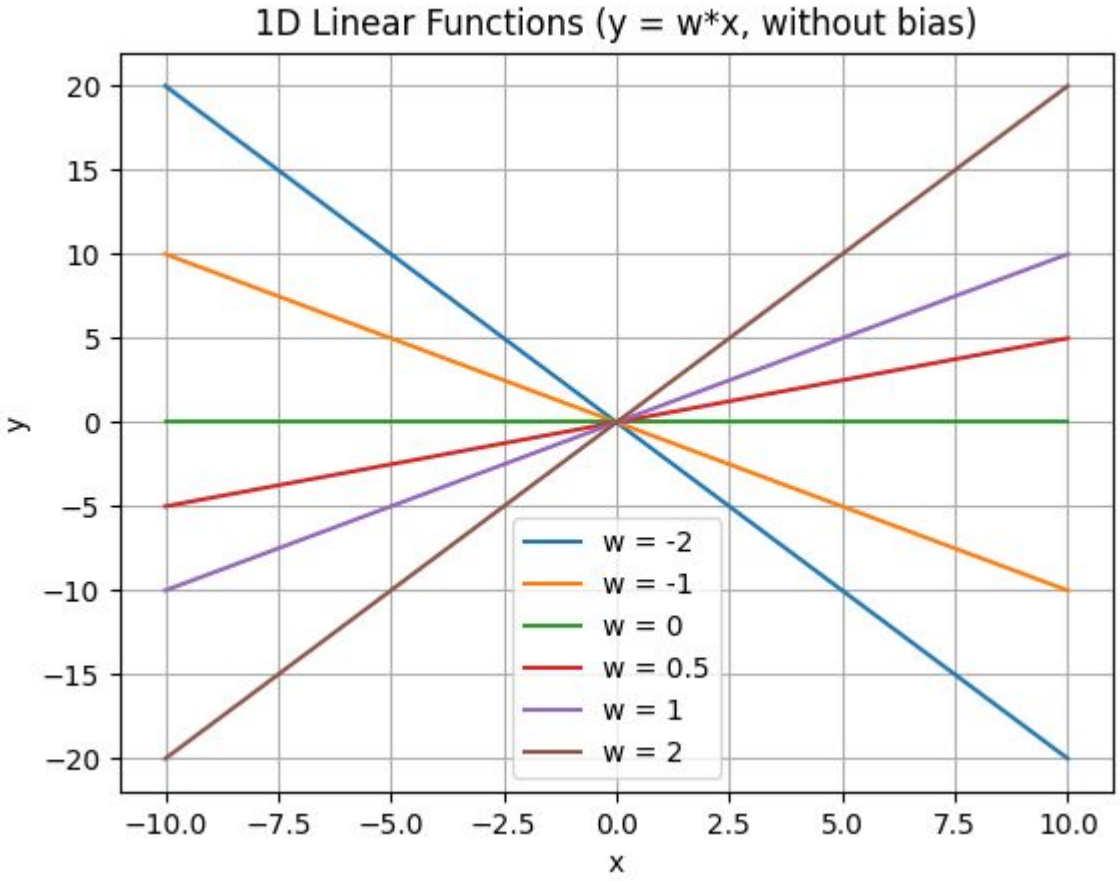
$$f(x) = f_L \circ f_{L-1} \circ \cdots \circ f_2 \circ f_1(x),$$

- More Functions:
  - Activation Functions
  - Objective Functions
  - Optimization Functions

Linear Functions

y = wx

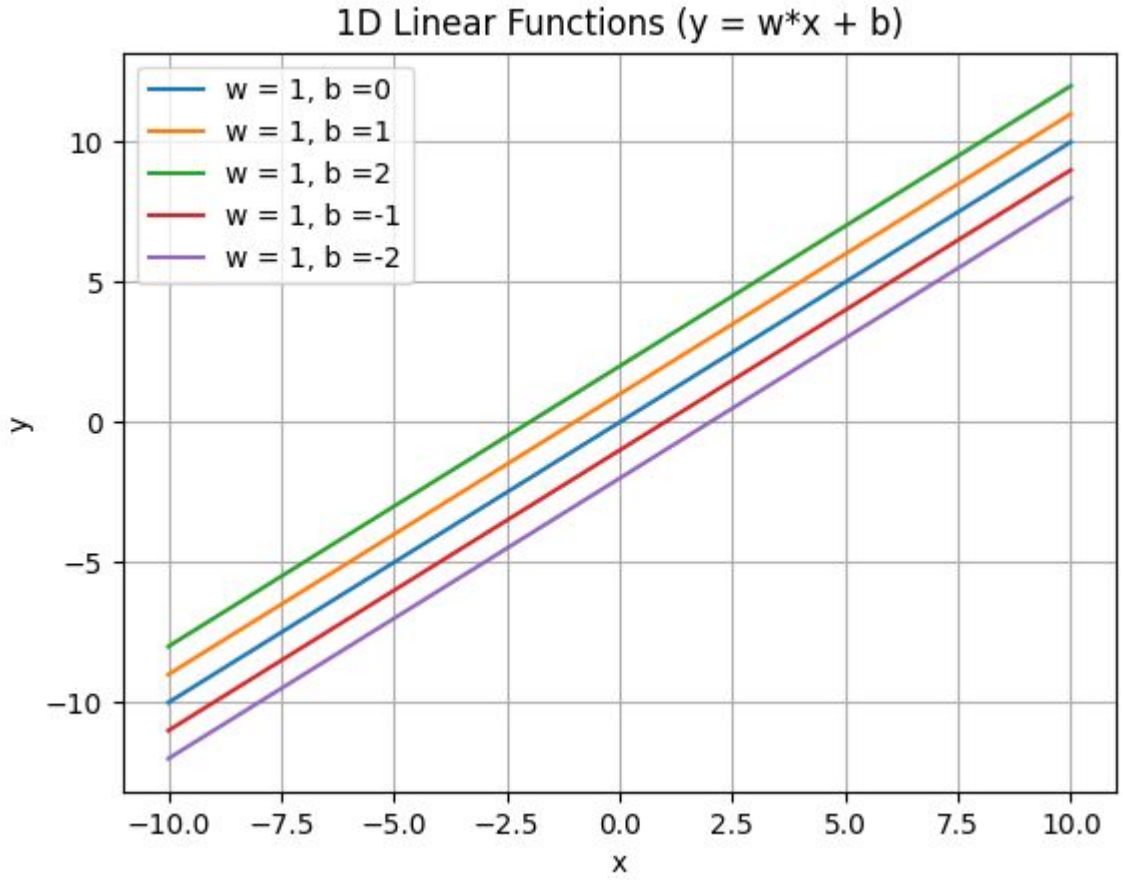
- **w (weight)** controls the slope
- since there is **no bias (b = 0)**, the line always passes through the origin (0,0).



Linear Functions

y = wx + b

- **b (bias)** → shifts the line up/down
- When **b ≠ 0**, the line **does NOT pass through origin**

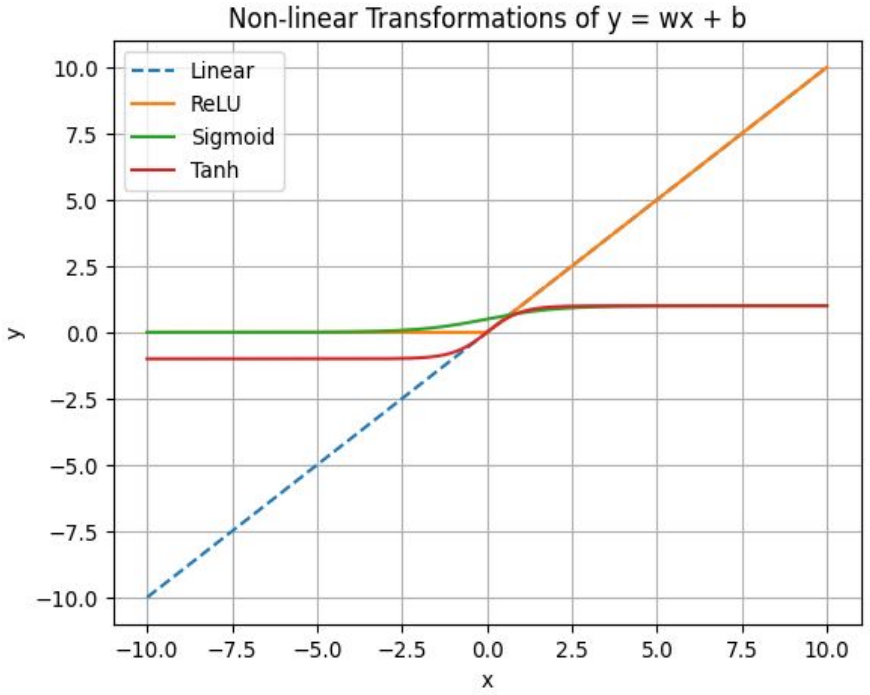


Non-Linear Functions

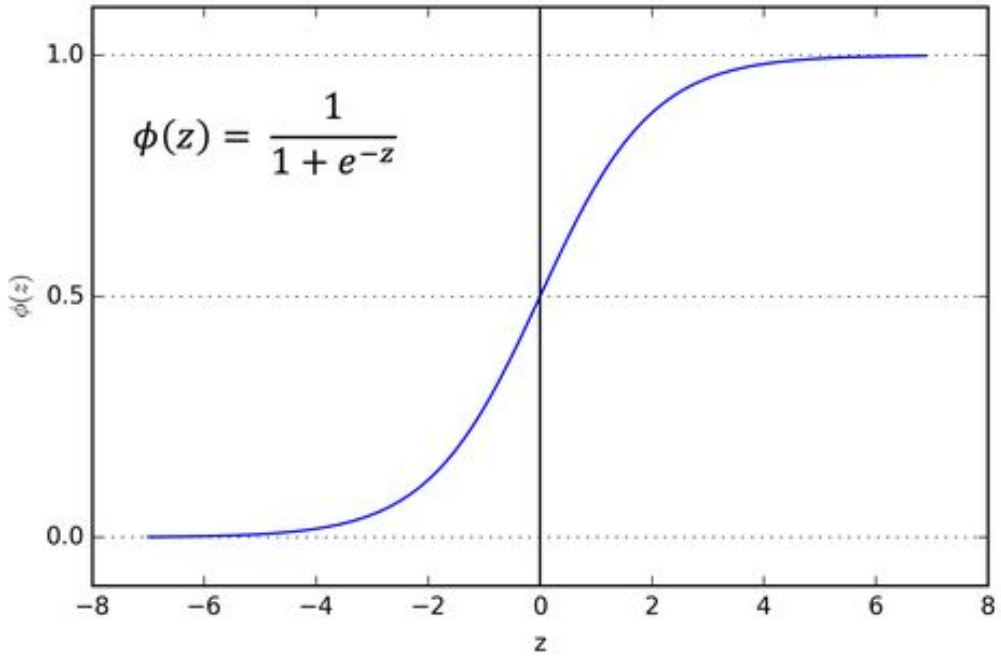
$$\text{ReLU:} \quad f(x) = \max(0,\, wx + b)$$

$$\text{Sigmoid:} \quad f(x) = \frac{1}{1 + e^{-(wx+b)}}$$

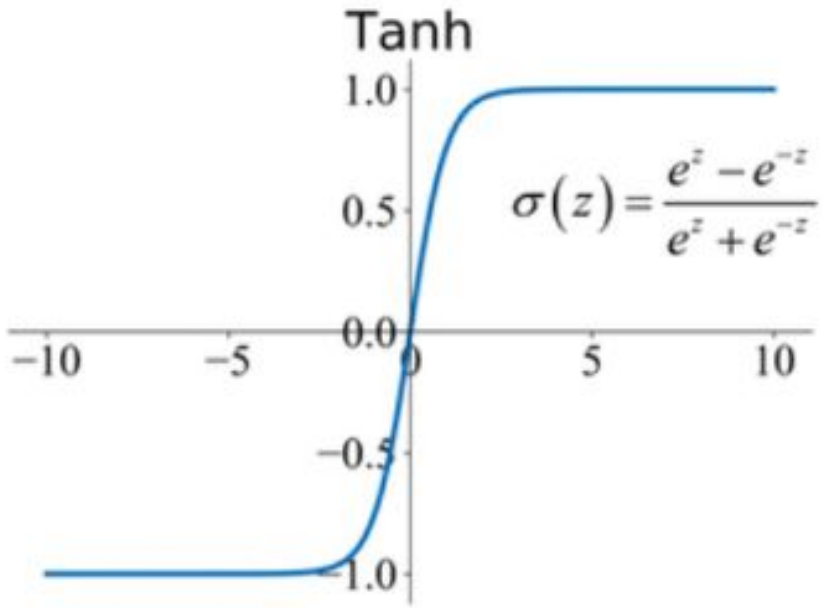
$$\text{Tanh:} \quad f(x) = \tanh(wx + b) = \frac{e^{(wx+b)} - e^{-(wx+b)}}{e^{(wx+b)} + e^{-(wx+b)}}$$



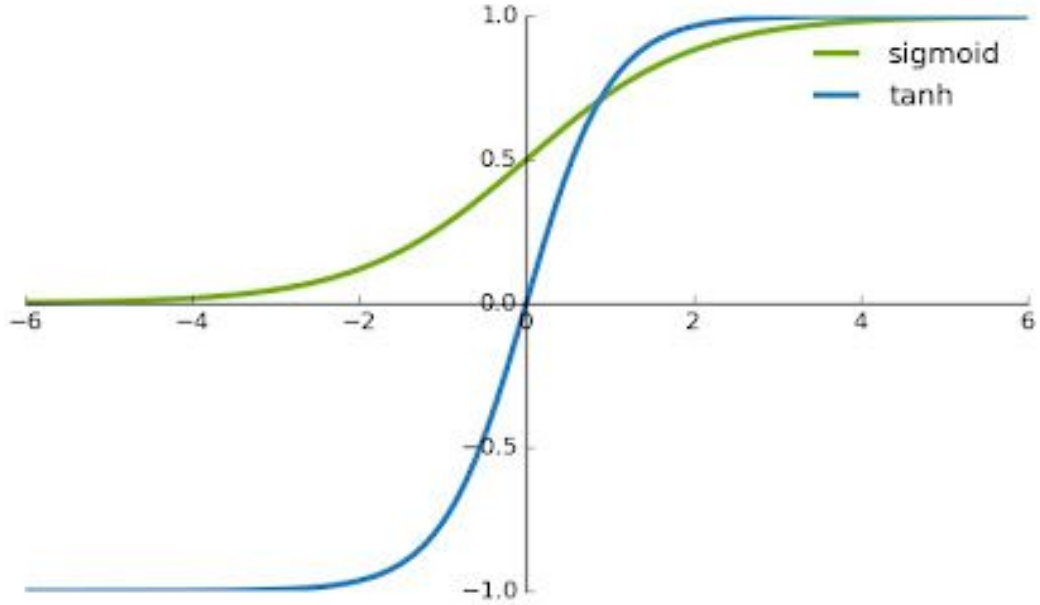
Non-Linear Function: Sigmoid



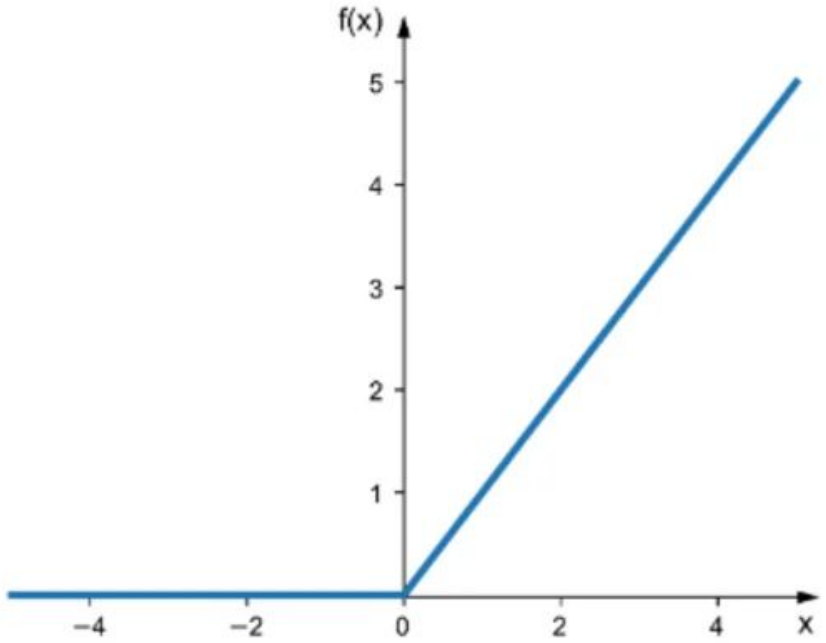
Non-Linear Function: Tanh



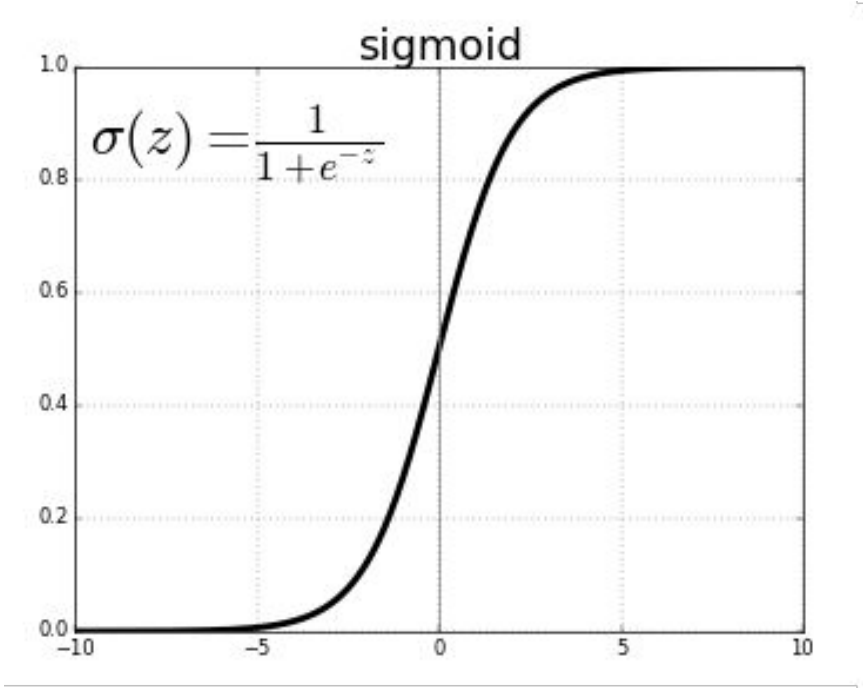
Non-Linear Function: Sigmoid Vs Tanh

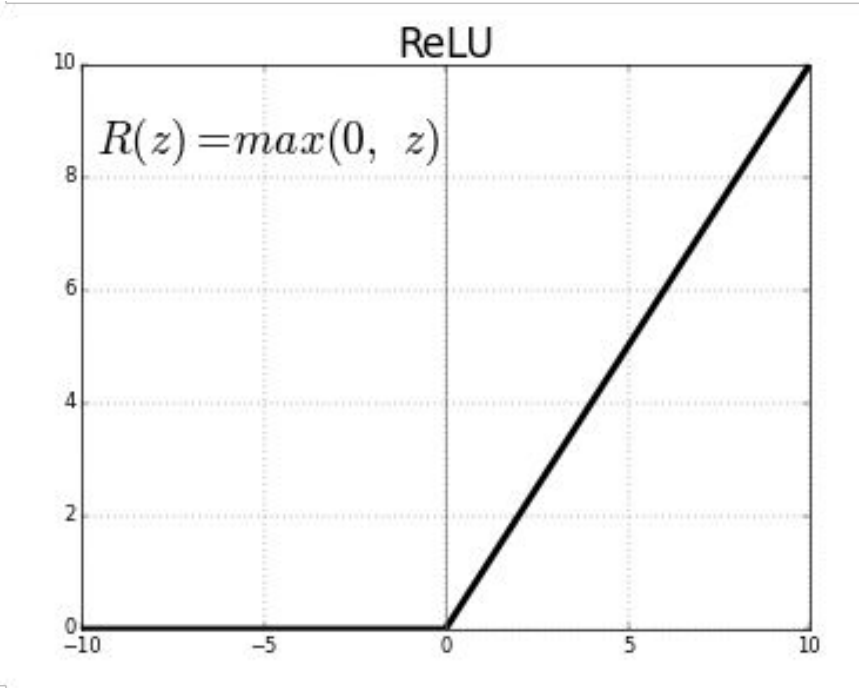


Non-Linear Function: ReLU



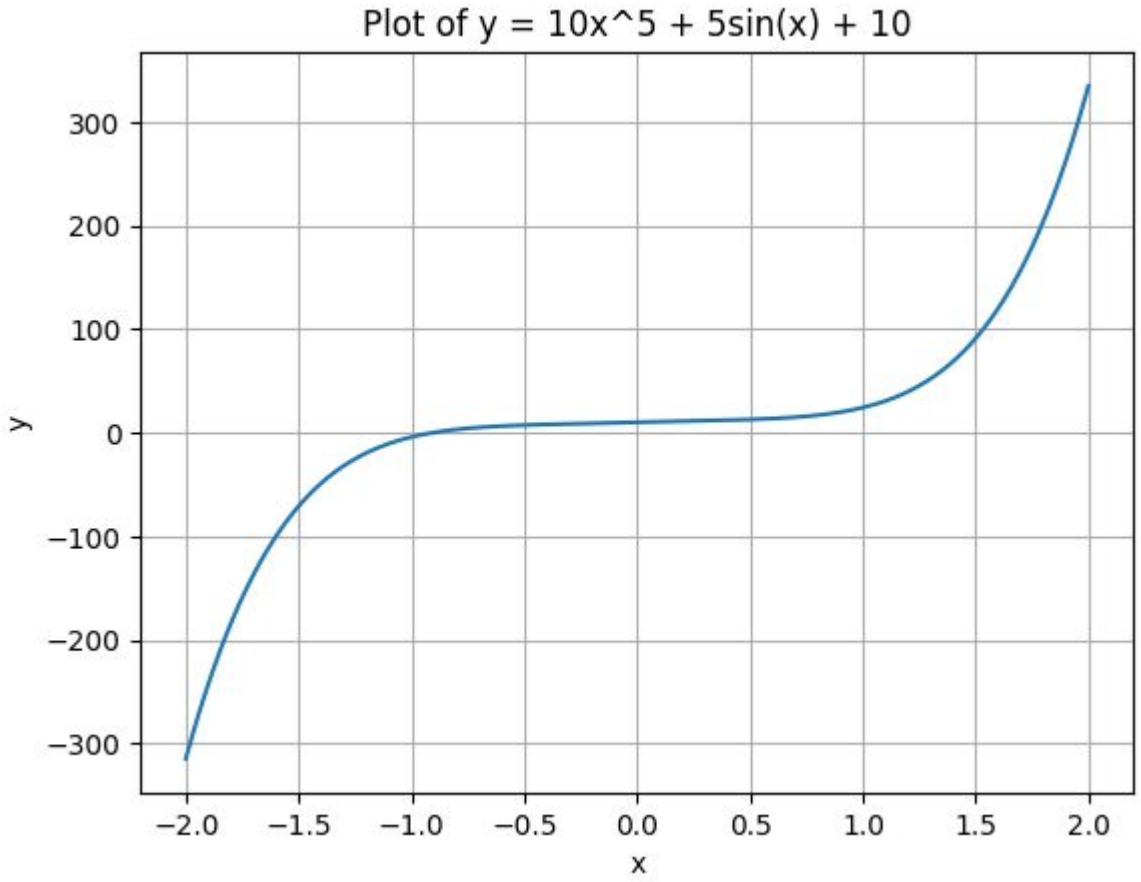
Non-Linear Function: Sigmoid Vs ReLU





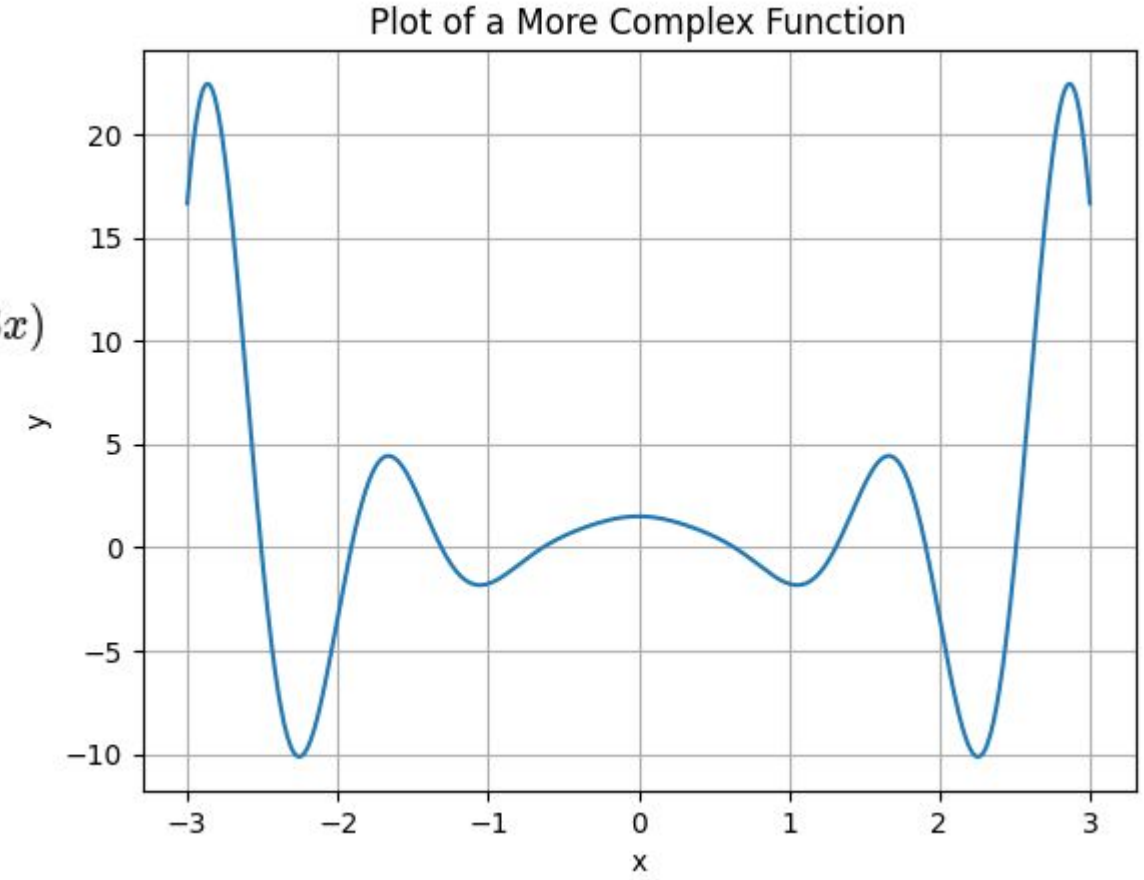
Non-Linear Function

$$y = 10x^5 + 5\sin(x) + 10$$

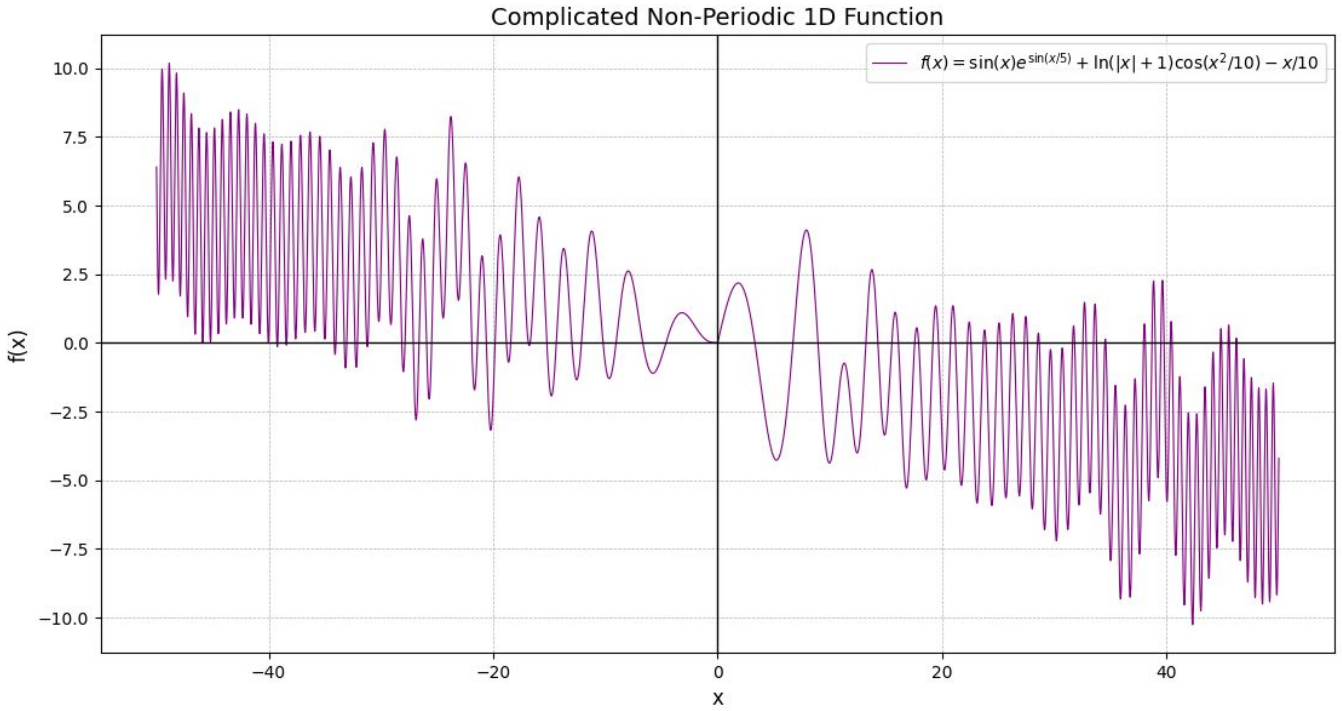


Non-Linear Function

$$y = x^3 \sin(5x) + 0.5e^{-x^2} + \cos(3x)$$



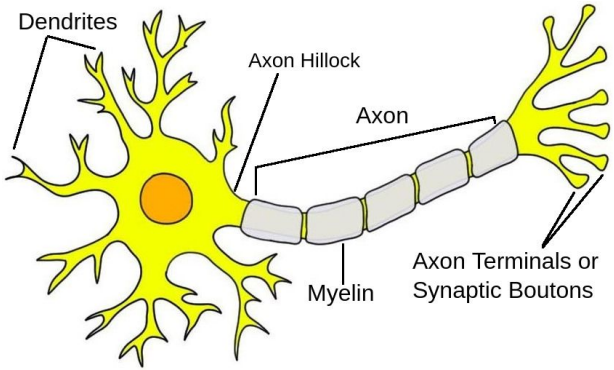
Very Complicated Non-Linear Functions



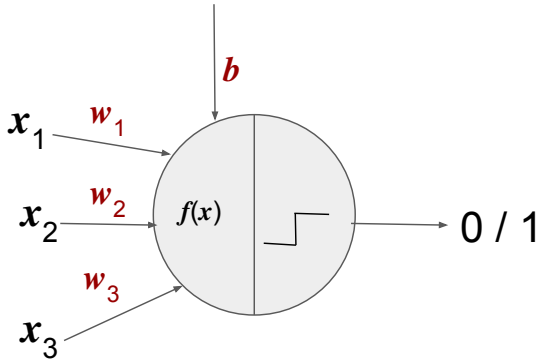
ANN

Perceptron

- A perceptron is a simple mathematical model of a biological neuron.
- Introduced by Frank Rosenblatt in 1957/1958.



Biological Neuron



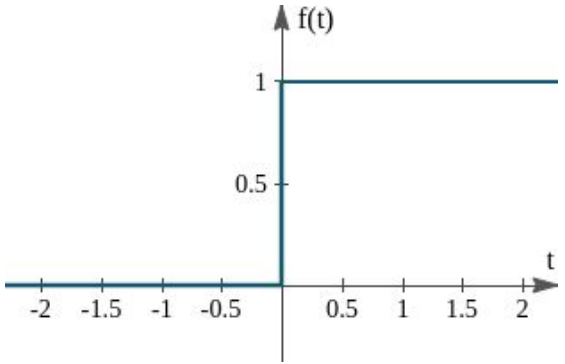
Perceptron



Frank Rosenblatt

Step Function in Perceptron

- Step function:
  - A nonlinear function
  - A piecewise constant function



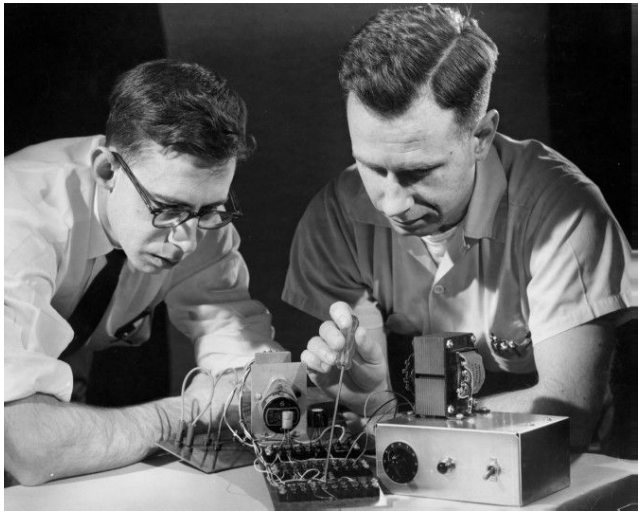
- In a Perceptron, step function is used as an activation function

$$\text{Let } z = \sum_{i=1}^{n} w_i \cdot x_i + b$$

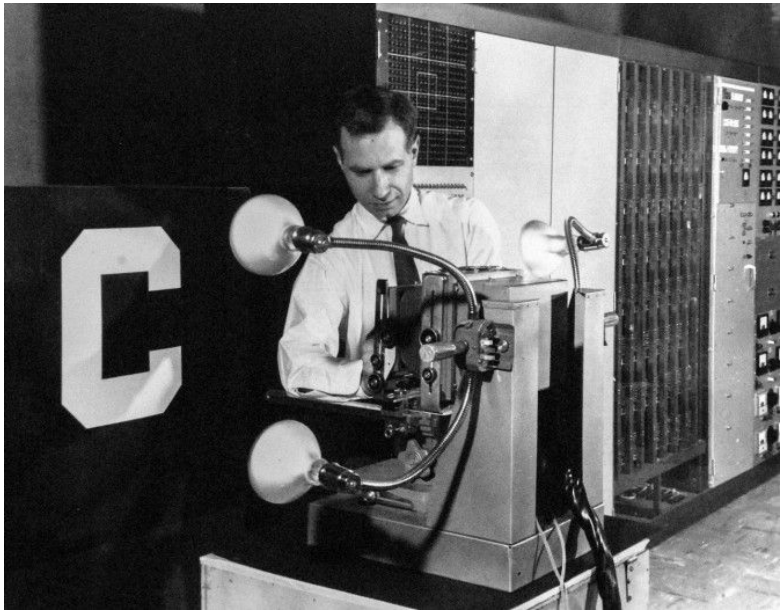
$$\text{Then, } y = f(z) = \begin{cases} 0, & \text{if } z \leq 0 \\ 1, & \text{if } z > 0 \end{cases}$$

where, w_i : i-th weight, x_i : i-th input, and b : bias

Perceptron as a Hardware



Frank Rosenblatt, left, and Charles W. Wightman work on part of the unit that became the first perceptron in December 1958.



Rosenblatt and the **Mark I Perceptron**

Artificial Neuron

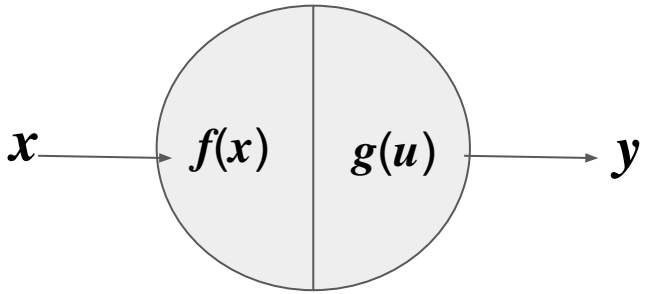
Artificial neuron is nothing but a composite function, $(g \circ f)(x)$

$$u = f(x)$$

$$y = g(u)$$

where

- x is input and y is output
- $f(x)$ is a linear function
- $g(u)$ is generally a nonlinear function



Artificial Neuron

Artificial Neuron

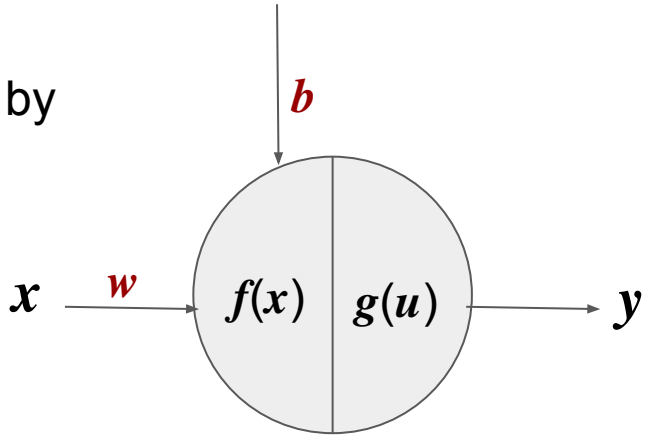
In an artificial neuron, at first input is multiplied by a weight and summed with bias.

$$y = g(f(x))$$

$$u = f(x) = wx + b$$

where

- w is weight which decides how much influence x will have on u.
- b is bias which ensures that u is not too big or too small on average



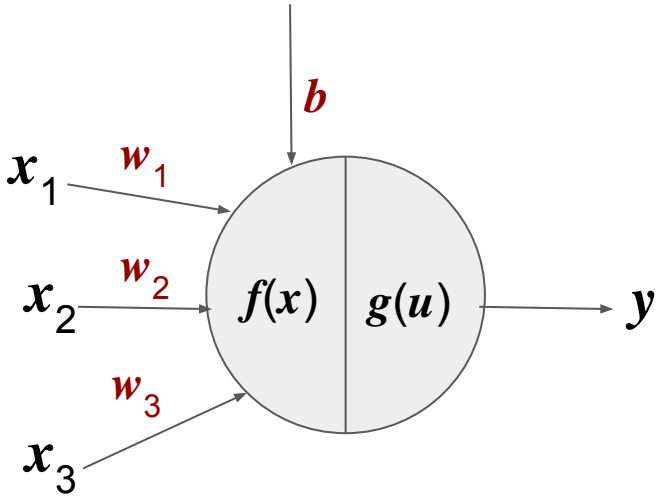
Artificial Neuron

Artificial Neuron

For a vector x, weight w is a vector and bias b is a scalar.

$$u = f(x)$$
$$= w_1 x_1 + w_2 x_2 + w_3 x_3 + b$$
$$= w^T x + b$$

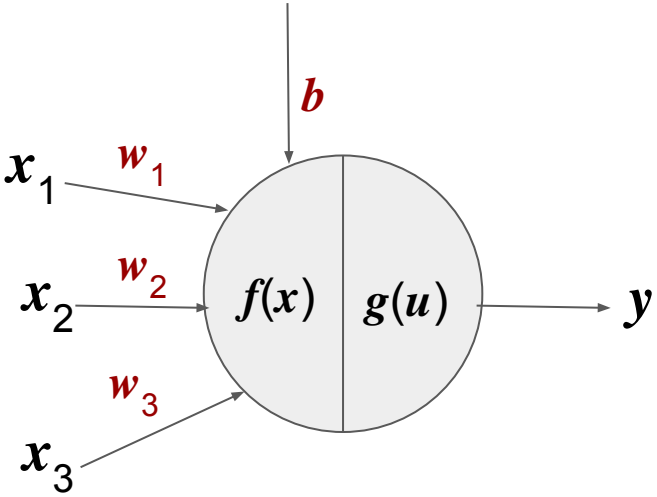
$g(u)$ is known as **activation function**. Generally nonlinear functions are used as activation functions.



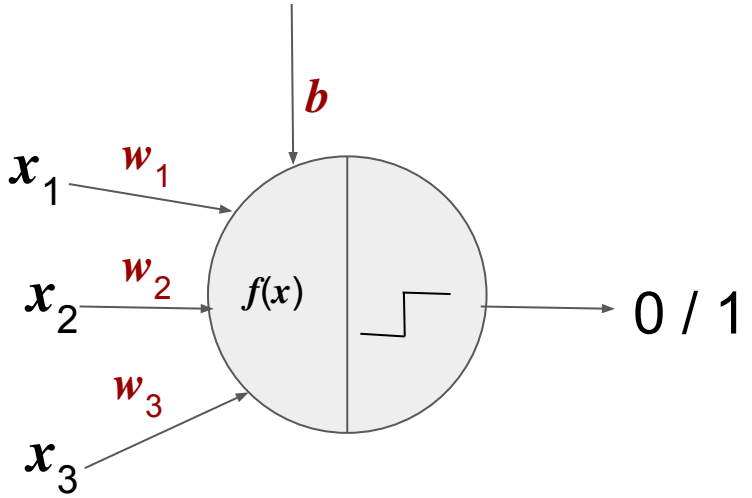
Artificial Neuron

Artificial Neuron vs Perceptron

- Perceptron is a specific form of an artificial neuron



Artificial Neuron



Perceptron

Artificial Neuron vs Perceptron

- An **artificial neuron** is a mathematical model inspired by a biological neuron. It's the fundamental building block of neural networks.

$$y = f\left(\sum w_i x_i + b\right)$$

  Activation function, $f(\cdot)$, of an artificial neuron can be **anything**: Sigmoid, ReLU, Tanh, Linear.

- The **perceptron** is a *very specific, early type* of artificial neuron proposed by Frank Rosenblatt (1957).

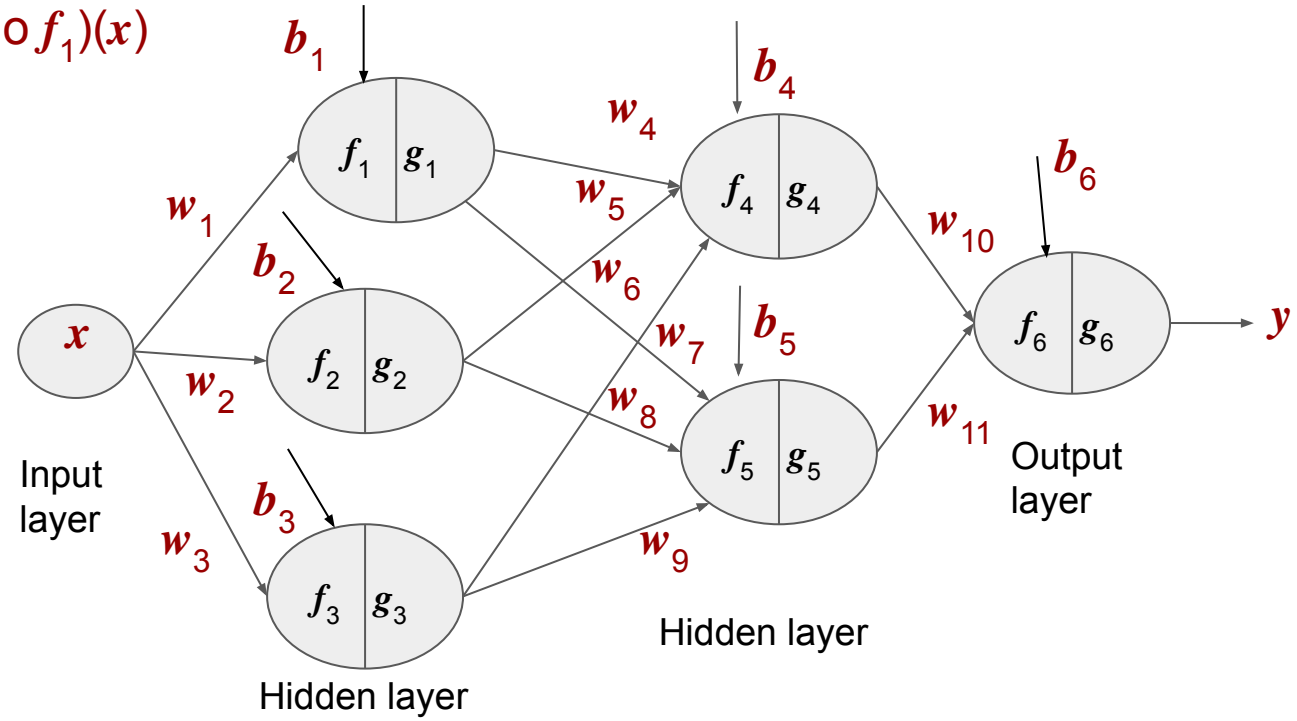
$$y = \text{step}\left(\sum w_i x_i + b\right)$$

Artificial Neural Network (ANN)

ANN is nothing but a composition of composite functions:

$(g_n \circ f_{n.....} g_2 \circ f_2 \circ g_1 \circ f_1)(x)$

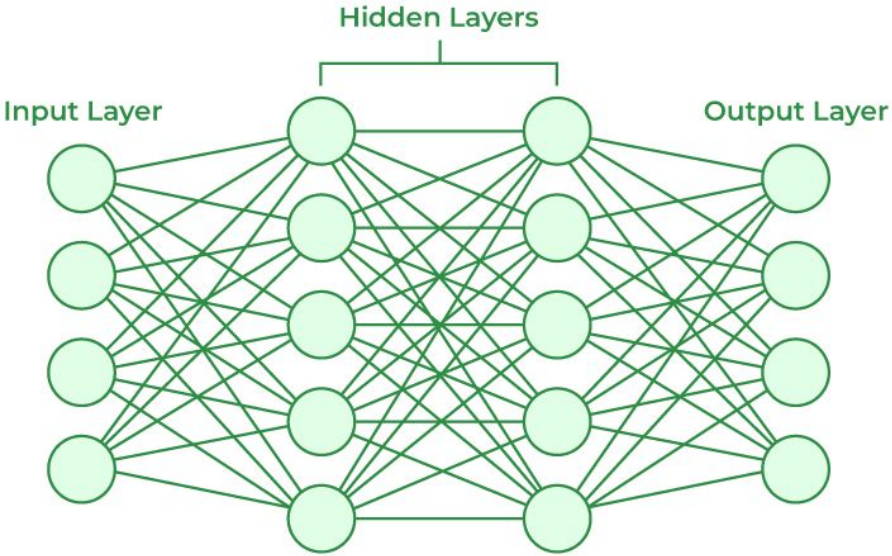
Number of parameters: weights + bias (optional) + any variables in $g(.)$ (optional)



Layers of ANN

All ANNs have three kinds of layers:

- **One** Input Layer: to receive data.
- **One or Multiple** Hidden Layers: to process data
- **One** Output layer: to produce the output.



Shallow ANN vs Deep ANN

- **Shallow ANN**
  - Shallow ANNs have one or two hidden layers.
  - They are suited for simpler tasks and small datasets, requiring less data and computation.
- **Deep ANN (DNN)**
  - They are in short known as DNN.
  - DNNs have multiple (often dozens or hundreds) hidden layers.
  - They can learn complex, hierarchical patterns and abstract features from large, high-dimensional datasets.
  - They are suitable for image recognition and natural language processing
  - Some DNNs:
    - GPT-3: 96 hidden layers
    - EfficientNet: 5-400 hidden layers
    - ResNet-152: 152 hidden layers

Hidden Layers

- Depth of an DNN = the number of hidden layers + the output layer
- Sometimes a deeper ANN has better performance than a shallower ANN.
- The number of hidden layers in a DNN depends on the complexity of the data:
  - **Linearly separable data**: No hidden layers are needed
  - **Less complex data**: 1–2 hidden layers are sufficient
  - **Large data**: 3–5 hidden layers are recommended
  - **Complex data**: Additional layers can be helpful
- A DNN has risks of
  - experiencing vanishing gradient problem
  - high sensitivity to input data
  - having similar performance of a shallow network.

Hidden Layers

- Increasing the number of hidden layers
  - Can increase the performance of the network.
- Adding too many hidden layers
  - Can lead to overfitting, where the model memorizes the training data but doesn't generalize well to new data.
- Reducing the number of hidden layers
  - Can directly impact the accuracy of the network. For complex problems, the network might not be trained properly with fewer hidden layers.
- Optimal number of layers depends on several factors including:
  - the data
  - the optimizer, and
  - the network's architecture

Number of Neurons

- Number of neurons in the:
  - input layer depends on the size of the input data
    - for 28x28 input image, 784 neurons in the input layer
  - output layer depends on the size of the output data
    - in a binary classifier, one or two neurons in the output layer
    - in a 10 class classifier, four or ten neurons in the output layer
  - hidden layers depends on us, i.e., who design the architecture of the NN
- More neurons means:
  - more parameters for tuning
  - more time for tuning parameters
  - more data for avoiding overfitting
  - more hardware support, i.e., computational power & memory during parameter tuning, and storage space for saving model.

Parameter Vs Hyperparameter

Parameters: which NN will learn from data

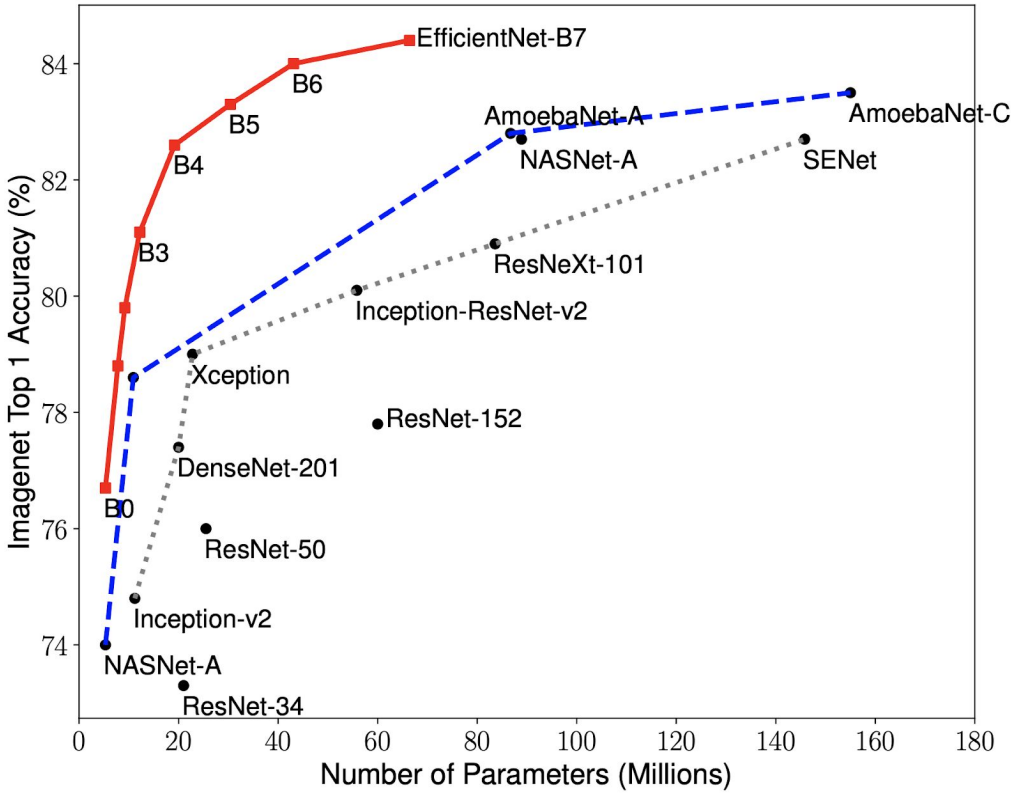
- w, b, any variables in $g(.)$

Hyperparameters: which we need to decide based on our experience, intuition or error-trial such as

- number of layers
- number of neurons in each layer
- activation function
- epoch number
- learning rate

Performance Vs Parameters

In general, a DNN having more parameters performs better



Keep in Mind

- Designing a suitable DNN automatically is not fully explored yet.
  - Recent Works: Neural Architecture Search—Finding the Best Model Design Automatically
- Designing a suitable network architecture is still a error-and-trial process.
- Not an architecture optimum for a dataset will be optimum for all datasets.
- It is upto us what will be the number of hidden layers and number of neurons in each hidden layer. However, we need to be careful.
- We should not design or work with any network which:
  - cannot be run in our available hardware support.
  - can be easily overfitted on our available training data.

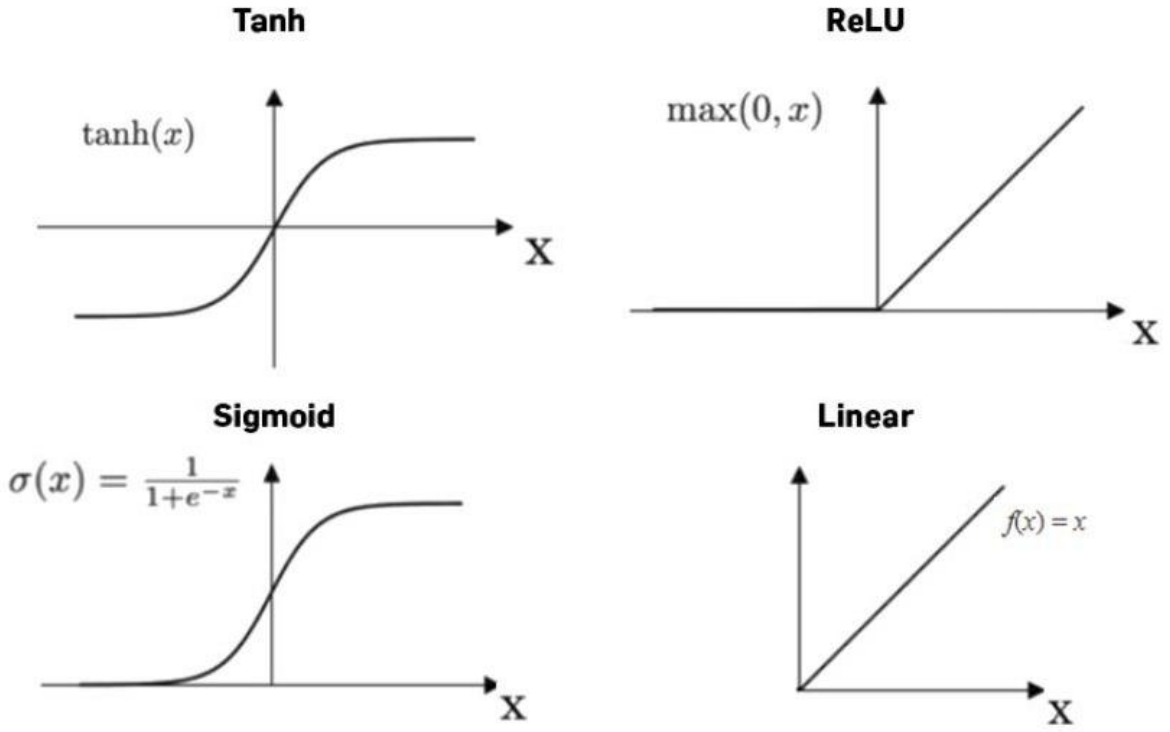
Activation Function

Activation Function

- An activation function is a mathematical equation that determines how much data should be passed from a neuron to the next neuron.
- It is the function that is applied on the weighted sum of the input of the neuron, i.e., it is $g(u)$.
- Generally, it is a nonlinear function for a neuron in the hidden layer and linear/non-linear function for neurons in the output layer.
- A linear activation function, also known as "no activation" or "identity function," is directly proportional to the input.

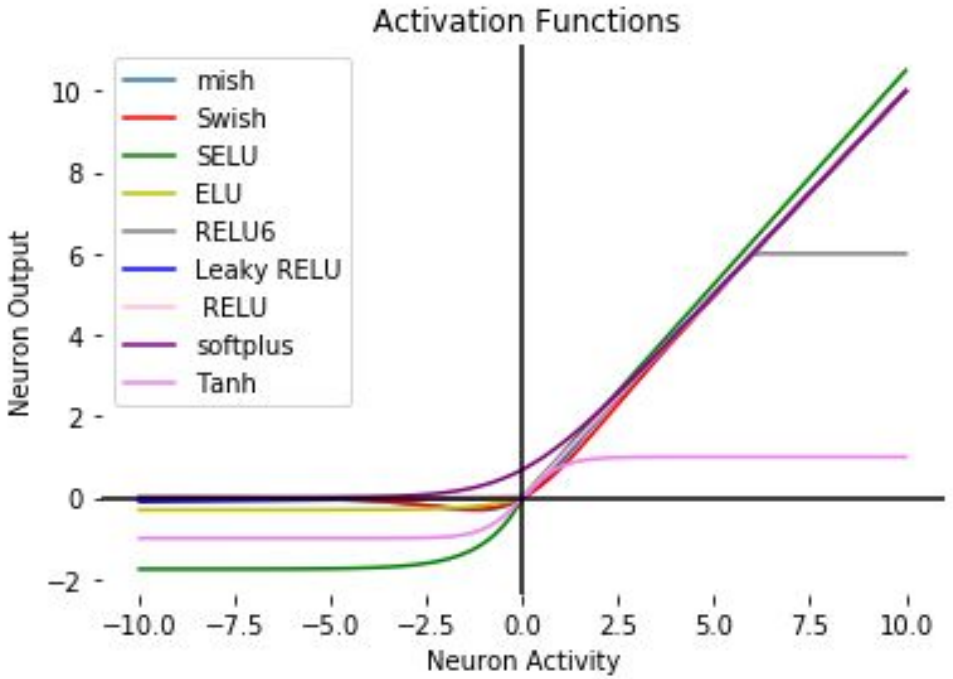
Activation Functions

- Linear activation function in the output layer generally used for regression problem.
- Depending on the range of output values, we need to choose activation function.

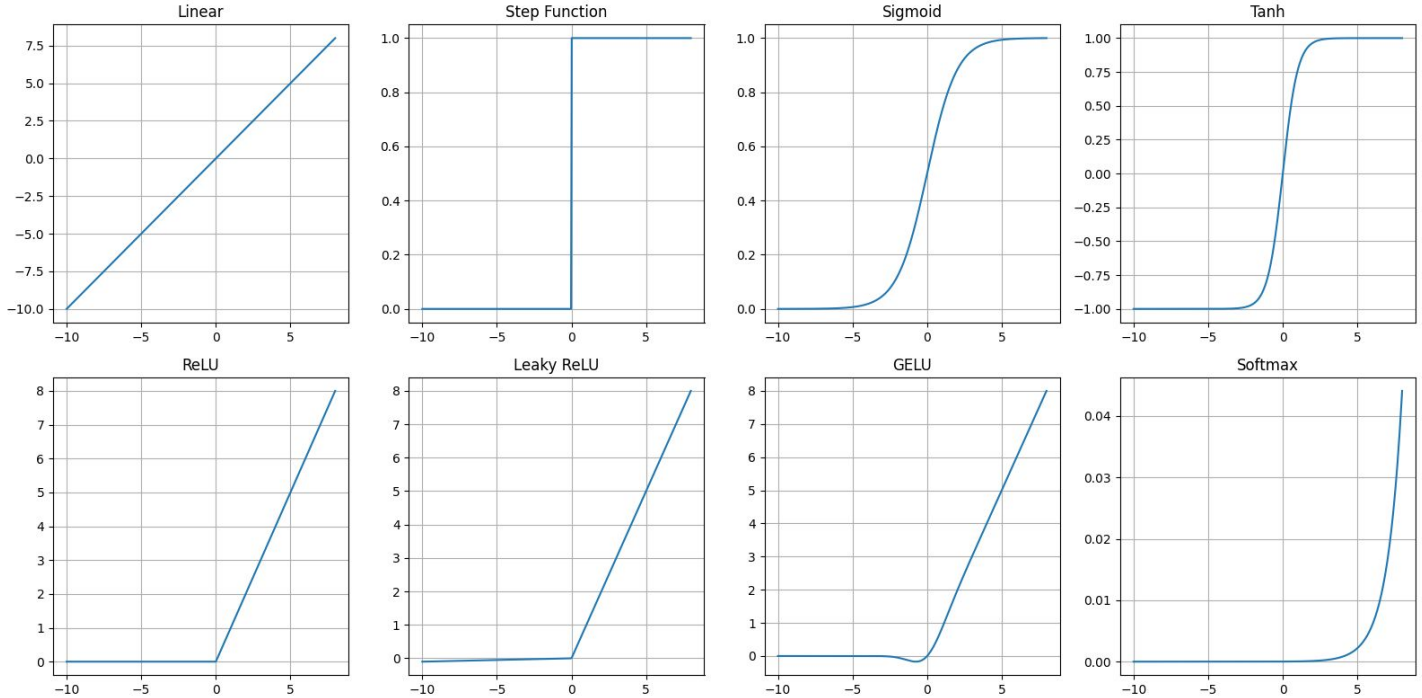


Activation Functions

LU (Linear Unit) family members such as ReLU, PReLU, ELU, etc., are quite popular as activation functions for the hidden layers.



Some Activation Functiuons



Activation Functions for Classification

In output layer, generally:

- Sigmoid is used for binary classification
- Softmax is used for multi-class classification

$$sigmoid,\ y_i = \frac{e^{x_i}}{1 + e^{x_i}}$$

$$softmax,\ y_i = \frac{e^{x_i}}{\sum_{j=1}^{k} e^{x_j}}$$

Both generate values in the range of 0-1.

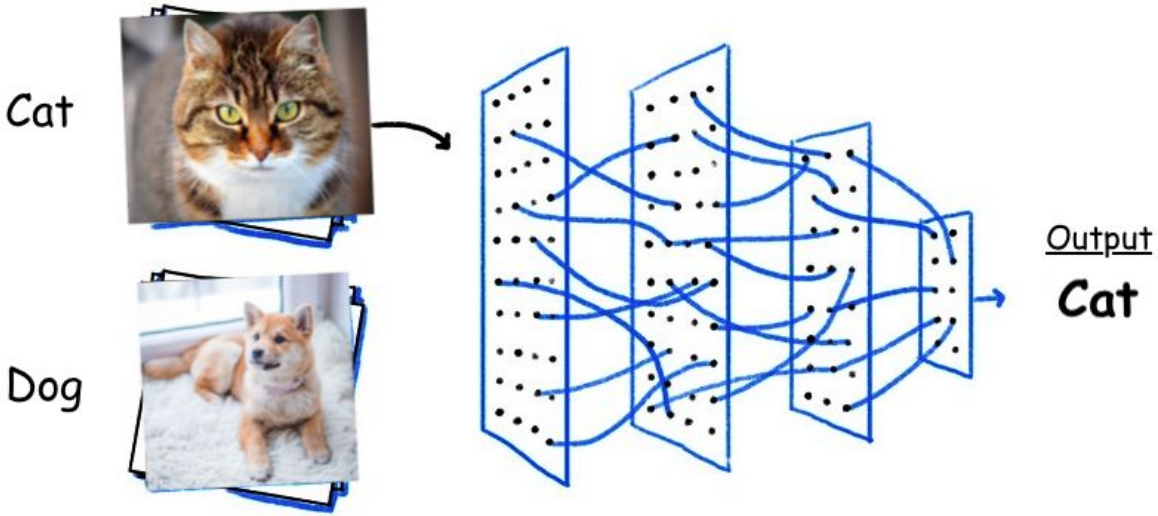
Summation of softmax values is 1.

Summation of sigmoid values in a classifier does not need to be 1.

Image Classification

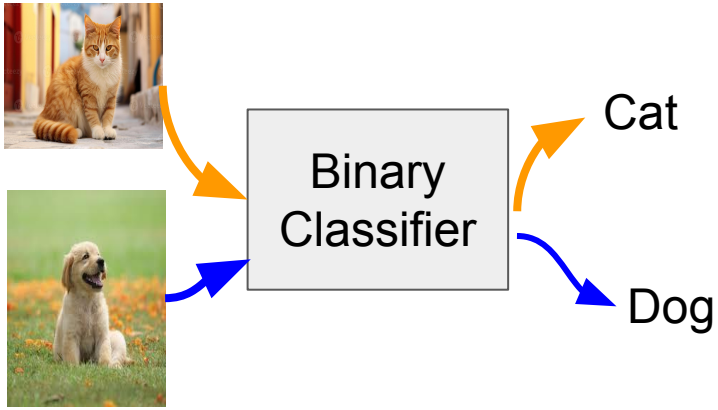
Image Classification

Image classification is a **computer vision** task in which an AI model assigns one or more pre-existing classes / categories / labels / groups to an image based on its visual content.

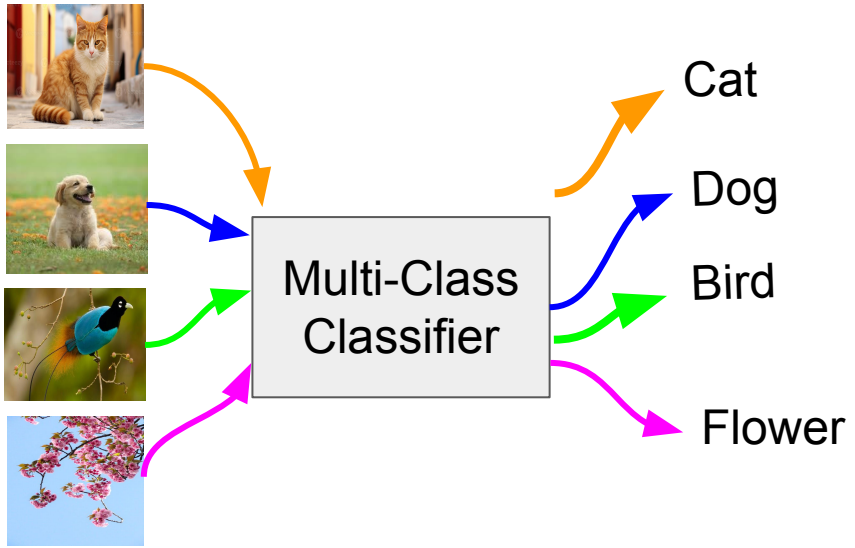


Types of Classifiers

- Depending on number of classes, classifiers can be: binary and multiclass.



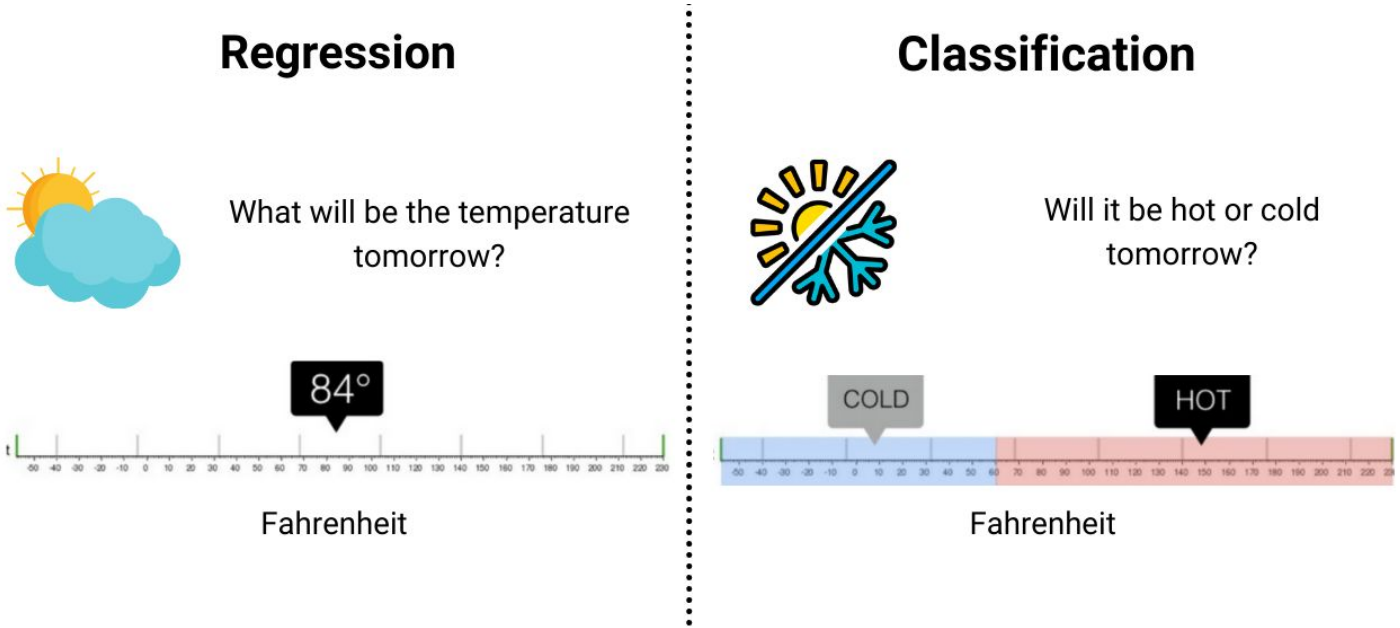
Number of classes = 2



Number of classes = 4

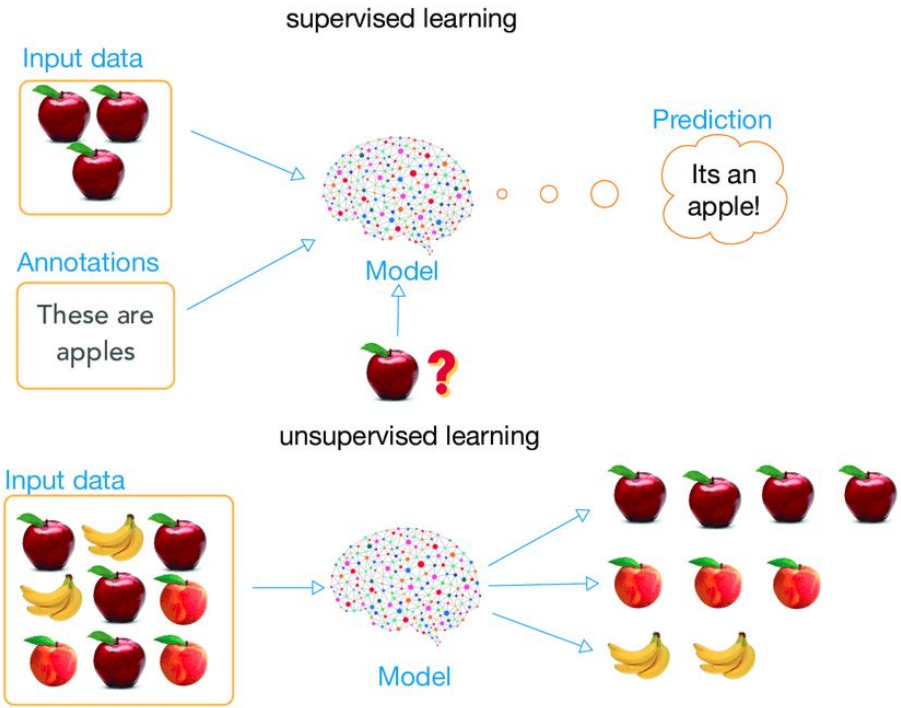
Regression Vs. Classification

Regression is a problem of predicting a continuous numeric value (e.g., a price, a temperature, a score) based on input data.



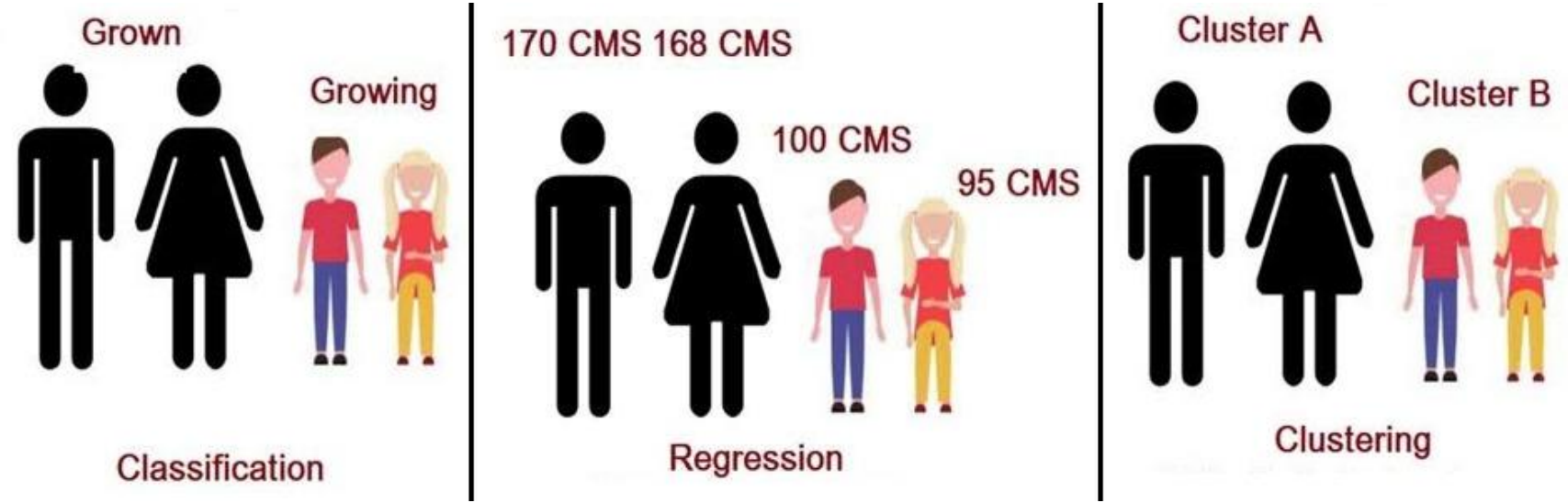
Supervised Learning Vs Unsupervised Learning

- Supervised learning uses labeled data
  - uses $\{x_n, y_n\}$
- Unsupervised learning uses unlabeled data
  - uses $\{x_n\}$



Clustering

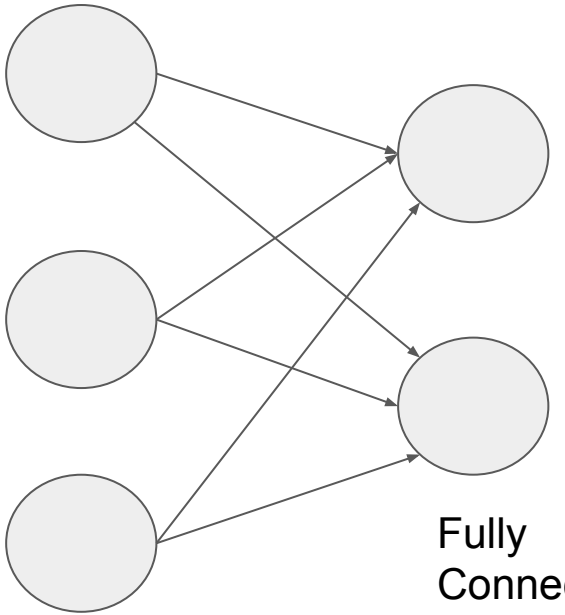
Clustering is an unsupervised learning technique designed to group unlabeled examples based on their similarity to each other.



MLP

Fully Connected Layer

- A fully connected layer connects every neuron in one layer to every neuron in its previous layer.
- An image need to be turned into a vector before feeding into an fully connected layer.
- Flatten() is used in Tensorflow.keras

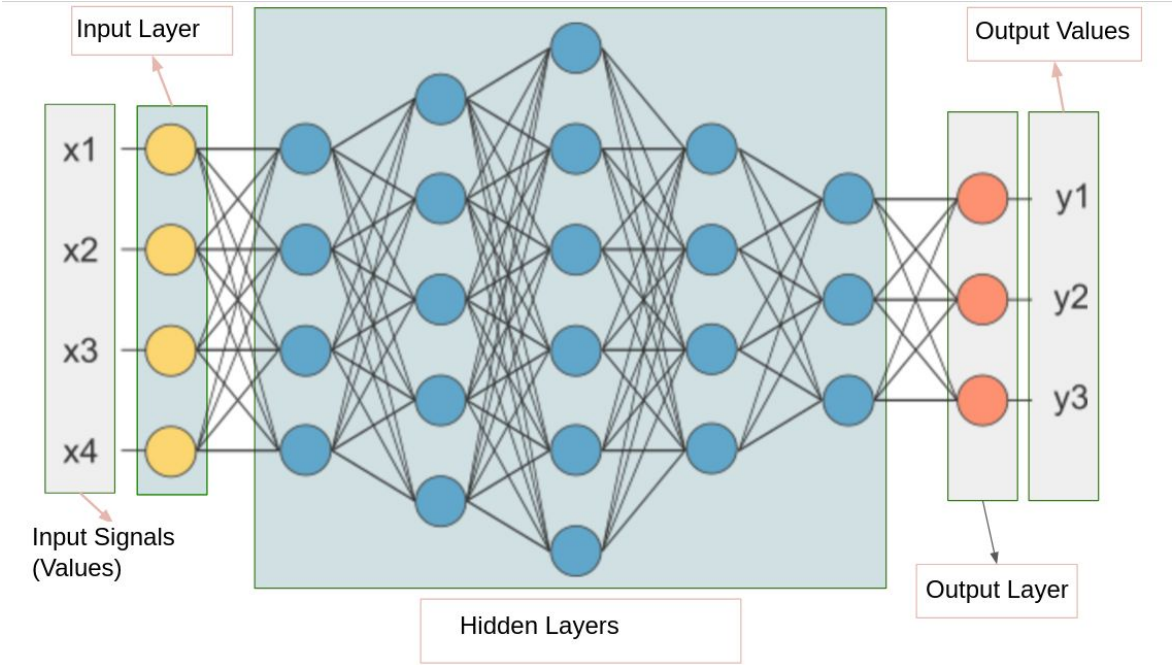


Fully Connected Layer

Fully Connected Feed-forward Neural Network (FCFNN)

FCFNN consists of a series of fully connected layers

- It is also known as multi-layer perceptron (MLP).

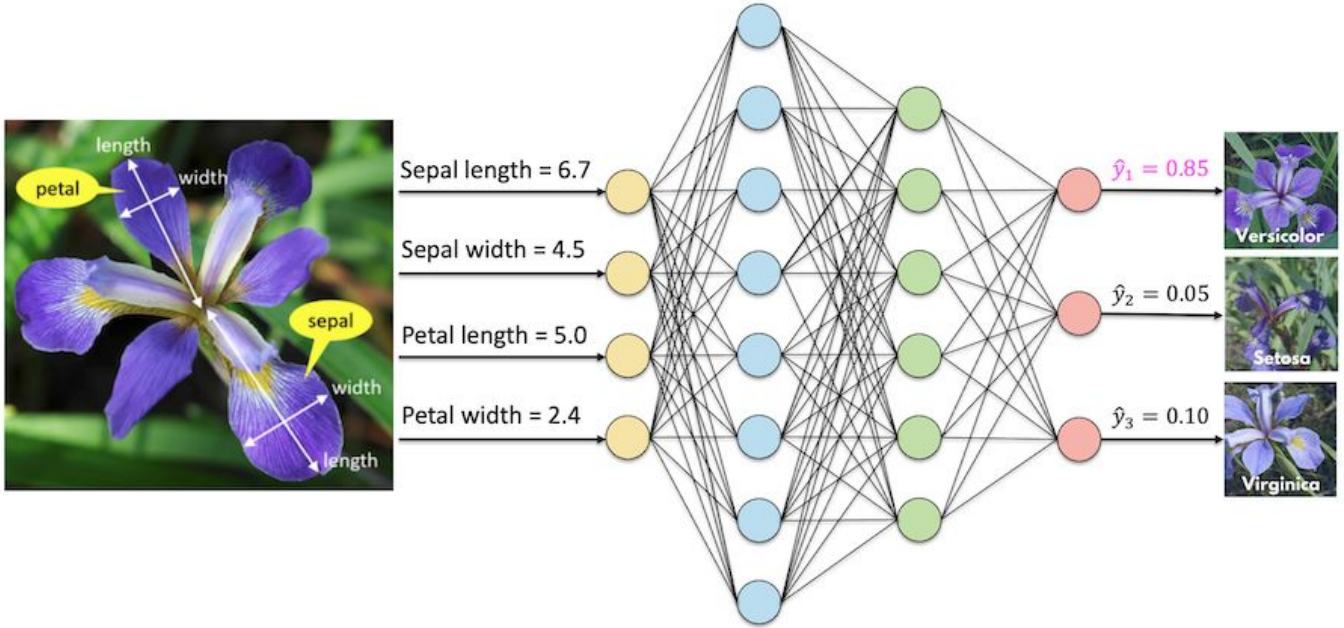


Multilayer Perceptron (MLP)

- A **multilayer perceptron (MLP)** is a type of **feedforward neural network** made of fully connected layers.
- It is **not really “perceptron-based” anymore.**
- It uses **artificial neurons generally with nonlinear activations**, not strict perceptrons.
- It can be used for:
  - Classification (e.g., MNIST digits)
  - Regression (predict continuous values)
  - Approximation of complex nonlinear functions

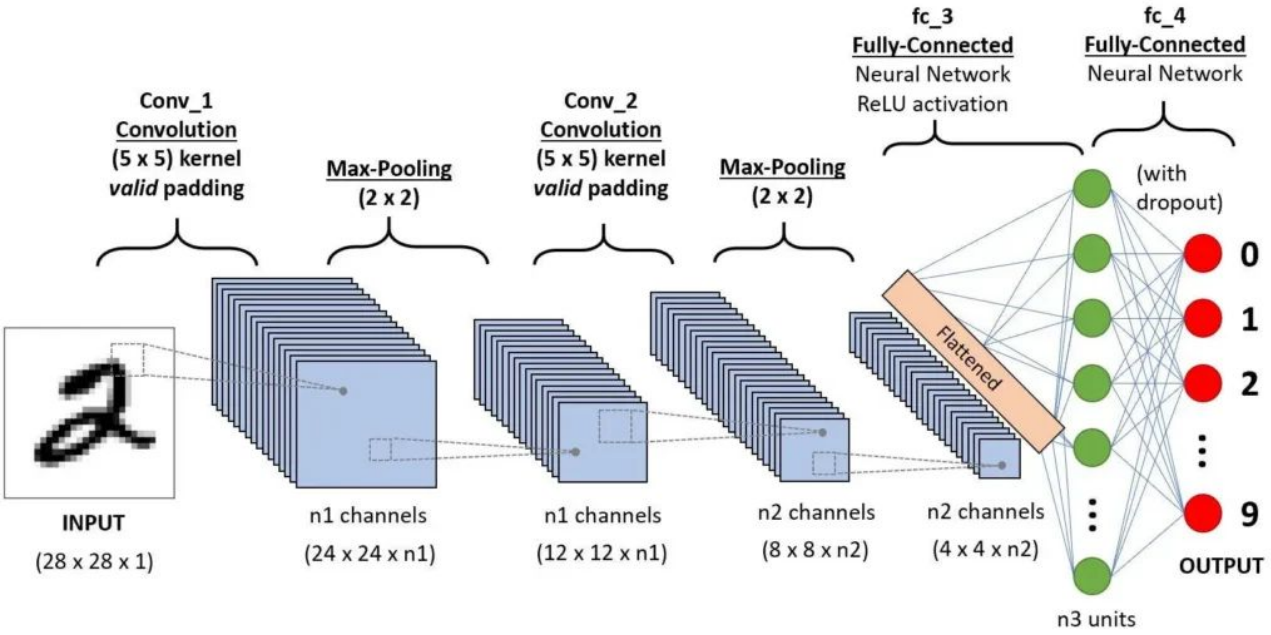
MLP as an Image Classifier

Generally extracted features are inserted into the MLP based classifier.



Convolutional Neural Network (CNN) as a Classifier

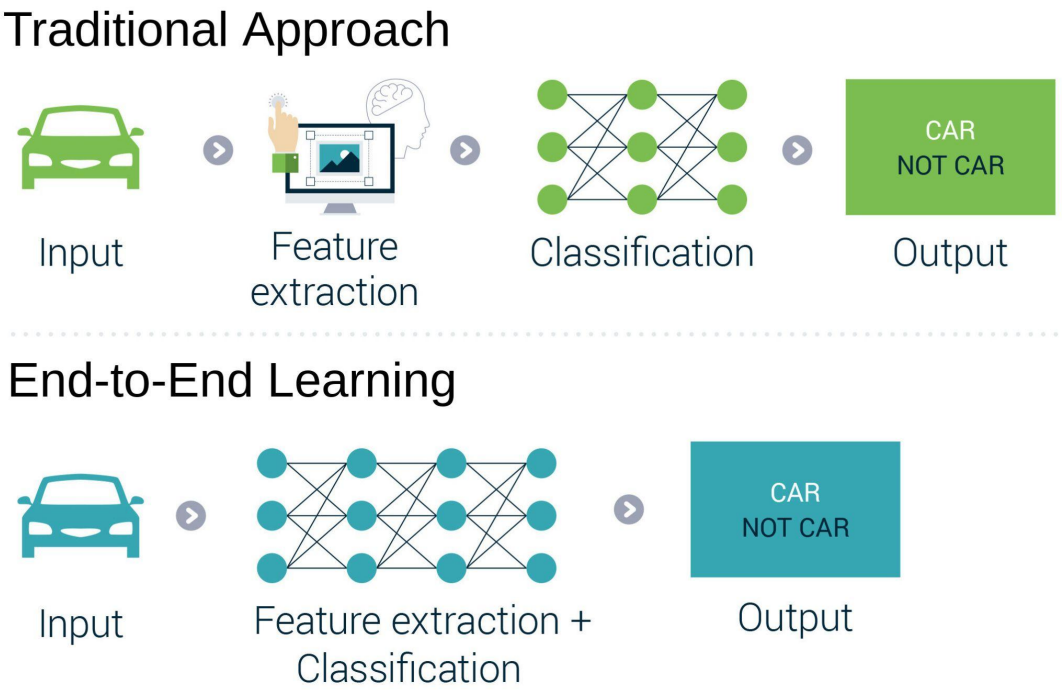
- CNN is a common and effective architecture for image classifier.
- It has convolutional layers are for feature extraction and an MLP for classification.



Training ANN

End-to-End Learning

- End-to-end learning trains a model to perform a task by learning the complete transformation from input to output without manual intermediate steps.
- Deep Learning often follows end-to-end learning.



Steps for Training & Testing

1. Prepare training set, validation set, and test set
2. Build a new model from scratch / Load a pretrained model / Build a model by adding a new head with a pre-trained backbone
3. Compile model
   model.compile(loss = loss_function, metrics = metric_list)
4. Train model
   model.fit(trainX, trainY, validation_split = 0.2, epochs = 100, callbacks = callbacks)
5. Evaluate model performance
   model.predict(testX)
   model.evaluate(testX, testY)

Training set, Validation Set, Test Set

- **Dataset Split:**
  - **Training Set:** used for finding optimum parameters
  - **Validation Set:** used for making decision i.e., finding optimum hyper-parameters
  - **Test Set:** used for evaluating model performance

- If any dataset is not split into training set, validation set and test set we need to shuffle whole dataset and manually split it into three parts:
  - Training Set: 70-80% of data
  - Validation set: 5-10% data
  - Test set: 10-20% data

Validation

A validation set is used to:

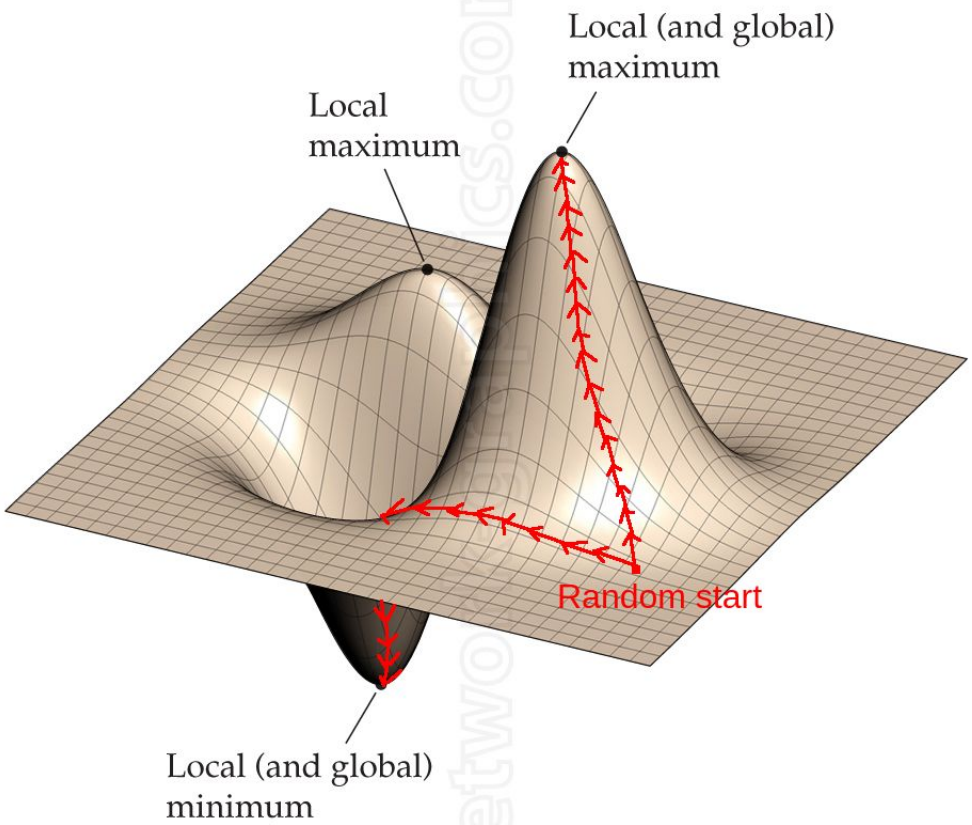
- Monitor model performance on **unseen data**
- Tune hyperparameters
- Decide when to stop training

Ways to use Validation Set:

- Split dataset once into:
  - Training set (e.g., 80% of data)
  - Validation set (e.g., 20% of data)
- Train model on training data
- Evaluate on validation data

Training Neural Network

- During training we try to find the parameter set for which loss is minimum or gain is maximum.
- Starting at random values, we update parameters by gradient descent algorithm.



Passes of Training DNN

There are two core passes used to **train a neural network**.

1. Forward Pass
    - The forward pass computes the **output and loss** from the input using current model parameters.
2. Backward Pass
    - The backward pass computes **gradients of the loss w.r.t parameters** using the chain rule.
    - Gradient descent is one of the most widely used and effective algorithms for updating parameters
    - Parameters are updated based on gradients.

Example

$y = wx + b$

Let:

- x = 2
- w = 3
- b = 1
- α = 0.1
- True, y = 5

Forward Pass:

$\hat{y} = 3.2 + 1$

$= 7$

Loss, L = mse

$= (\hat{y} - y)^2$

$= (wx + b - y)^2$

$= (7 - 5)^2$

$= 2^2$

$= 4$

Backward Pass:

$$L = (\hat{y} - y)^2 = (7 - 5)^2 = 4$$

$$\frac{\partial \hat{y}}{\partial w} = x = 2$$

$$\frac{\partial L}{\partial w} = 4 \cdot 2 = 8$$

$$\frac{\partial L}{\partial b} = 4 \cdot 1 = 4$$

$$w = 3 - 0.1 \cdot 8 = 2.2$$

$$b = 1 - 0.1 \cdot 4 = 0.6$$

One Hot Encoding

One-hot encoding is a technique used to convert **categorical data into numerical format** so that deep learning models can understand it.

Example:

- Suppose we have 3 classes: Cat, Dog and Bird
- Label Encoding: Cat → 0, Dog → 1, Bird → 2
- **One Hot Encoding:**

  Cat — [0, 0, 0]

  Dog — [0, 1, 0]

  Bird — [0, 0, 1]



Objective Function

Objective Functions

An **objective function** is the function that measures **how well a model performs**, and the goal is to adjust parameters to achieve the **optimal (minimum or maximum) value** during training.

Typically written as:

$$\mathcal{L}(w) = \text{Loss function}$$

or more generally: $\min_{w} \mathcal{L}(w)$

- $w \rightarrow$ model parameters (weights, biases)
- $\mathcal{L} \rightarrow$ error or loss

Objective vs Loss vs Cost

- The terms **objective function**, **loss function**, and **cost function** are closely related, but they are **not exactly the same**.
- In deep learning, they are often used interchangeably, although each has a more precise meaning.

Term	Meaning
Loss	Error for one sample
Cost	Average loss over dataset
Objective	What we optimize (may include regularization)

Types of Objective Functions

1. **Minimization (Most Common)**
   a. Reduce Error
   b. Example:

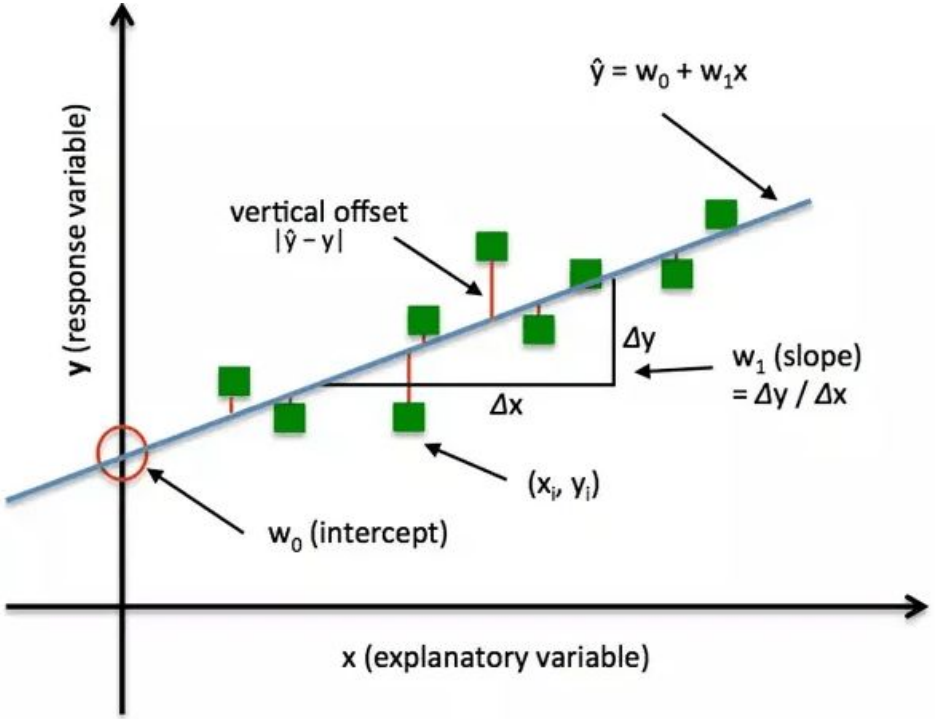
$$\min \text{ MSE} = \frac{1}{n}\sum(y_{true} - y_{pred})^2$$

2. **Maximization**
   a. Maximize reward or likelihood
   b. Example:

$$\max \log P(\text{data})$$

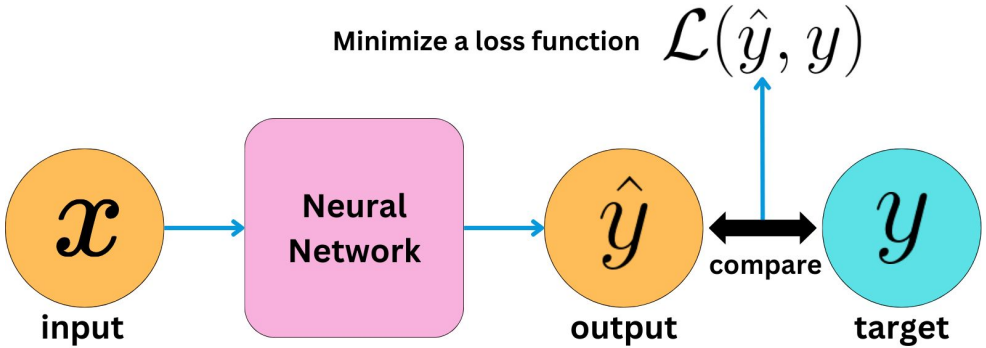
Loss

- Error is a measurement of the difference between the predicted output of the model and the actual output.
- Loss is the weighted error.
- Different types of loss:
  - Absolute loss: $|\hat{y} - y|$
  - Squared loss: $(\hat{y} - y)^2$
  - Log loss: $- y.\log(\hat{y})$



Loss Function

- A loss function is a function which estimates the total loss of all data samples.
- It is used to find optimum values of parameters of a neural network during training.
- It is a function of parameters.
- Different ways of writing:
  - J(ω); L(θ|x); $\ell(\omega)$; $\mathcal{L}$(ŷ - y);



Loss — a Function of Parameters (not x)

$y = b + wx = f(b, w, x)$

$y = w_0 + w_1 x = f(w_0, w_1, x)$

$y = 10 + 5x = f(x)$

$y = w_0 + w_1 100 = f(w_0, w_1)$

A model takes input x and produces output:

$$\hat{y} = f(x; w)$$

- $x \rightarrow$ input (fixed data)
- $w \rightarrow$ parameters (what we learn)

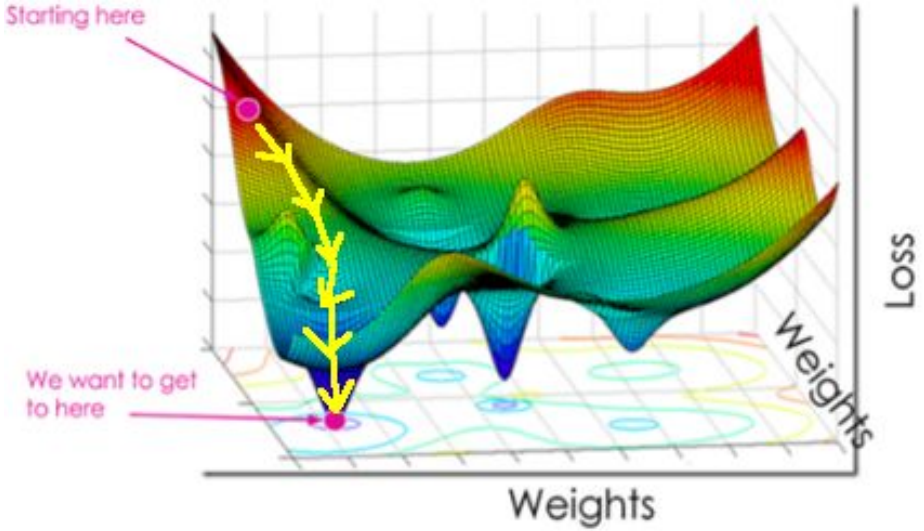
$$\mathcal{L}(w) = \ell(y, \hat{y}) = \ell(y, f(x; w))$$

Here:

- x and y are **given (constants)**
- Only w changes

Training Neural Network

Starting at random values, we update parameters so that we can get the minimum loss by gradient descent algorithm.



Different Loss Functions

Different loss functions penalizes wrong predictions differently

Popular losses:

- Regression:
  - Mean-Squared Error (MSE)
  - Mean Absolute Error (MAE)

- Classification:
  - Binary Cross-Entropy
  - Categorical Cross-Entropy
  - Focal Cross-Entropy
  - Hinge Loss

L1 & L2 Loss

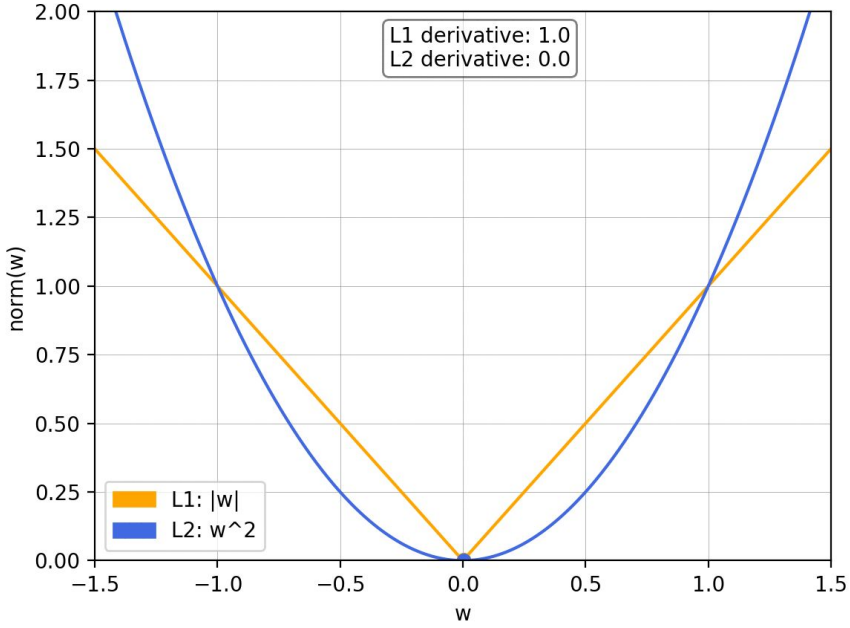
- Absolute Loss is also known as L1 loss
- Squared Error Loss is also known as L2 loss

$$L_1 = \sum_{i=1}^{n} |y_{gt} - y_{pred}|$$

$$L_2 = \sum_{i=1}^{n} (y_{gt} - y_{pred})^2$$

$$MAE = \frac{1}{n} \sum_{i=1}^{n} |y_{gt} - y_{pred}|$$

$$MSE = \frac{1}{n} \sum_{i=1}^{n} (y_{gt} - y_{pred})^2$$



Entropy

- It measures the average uncertainty or information content of a random variable.
- It quantifies how unpredictable a distribution is.
- A lower entropy indicates higher predictability (less uncertainty), while a higher entropy indicates more unpredictability.

$$\text{Entropy, } H(p) = -\sum_x p(x) \log p(x)$$

$$\text{Expected Value, } \mathbb{E}[X] = \sum_x x\, p(x)$$

Cross Entropy (CE)

- It quantifies how well a predicted probability distribution matches the true distribution of the data.
- It is often used as a loss function in classification tasks, aiming to minimize the difference between the predicted and true distributions.

Cross Entropy, $H(p, q) = -\sum_x p(x) \log q(x)$

Here:

- $p(x)$ = true distribution
- $q(x)$ = predicted distribution

Entropy, $H(p) = -\sum_x p(x) \log p(x)$

Expected Value, $\mathbb{E}[X] = \sum_x x\, p(x)$

Categorical Cross Entropy

Categorical Cross Entropy is the standard loss used for **multi-class classification** (including semantic segmentation) when the ground truth is **one-hot encoded**.

$$L = -\sum_{i=1}^{C} y_i \log(\hat{y}_i)$$

- C: number of classes
- y_i: true label (one-hot: 0 or 1)
- $\hat{y}_i$: predicted probability (from **softmax**)

Binary Cross Entropy

Binary Cross Entropy (BCE) is used for **binary classification** (two classes) or **per-pixel binary segmentation** (e.g., foreground vs background).

$$L = -\left[y \log(p) + (1 - y) \log(1 - p)\right]$$

- $y \in \{0, 1\}$: true label
- p: predicted probability (after sigmoid)

- If $y = 1 \rightarrow$ loss = $-\log(p)$
- If $y = 0 \rightarrow$ loss = $-\log(1 - p)$

The model is penalized when it assigns **low probability to the correct class**.

Sparse-CE vs Categorical CE

Both are loss functions used for **multi-class classification** (including **semantic segmentation**), but they differ in **how the ground truth labels are represented**.

- Sparse Categorical Crossentropy: integer labels
- Categorical Crossentropy: one-hot encoded vectors

Common Mistake

- Sparse loss + one-hot labels
- Categorical loss + integer labels

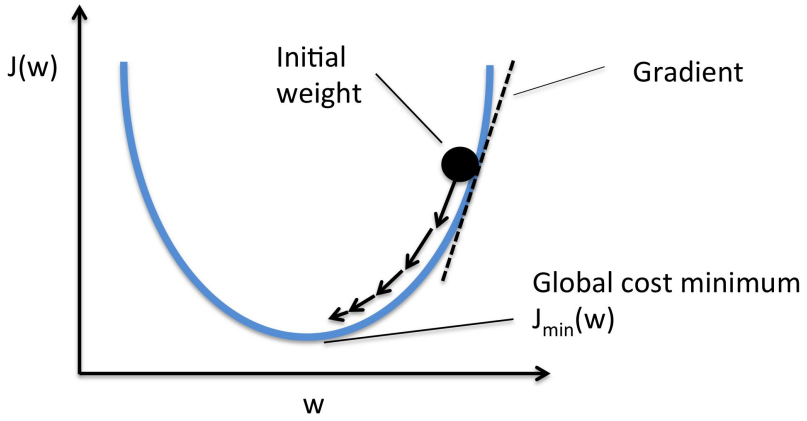
Gradient Descent Algorithm

Gradient Descent Algorithm

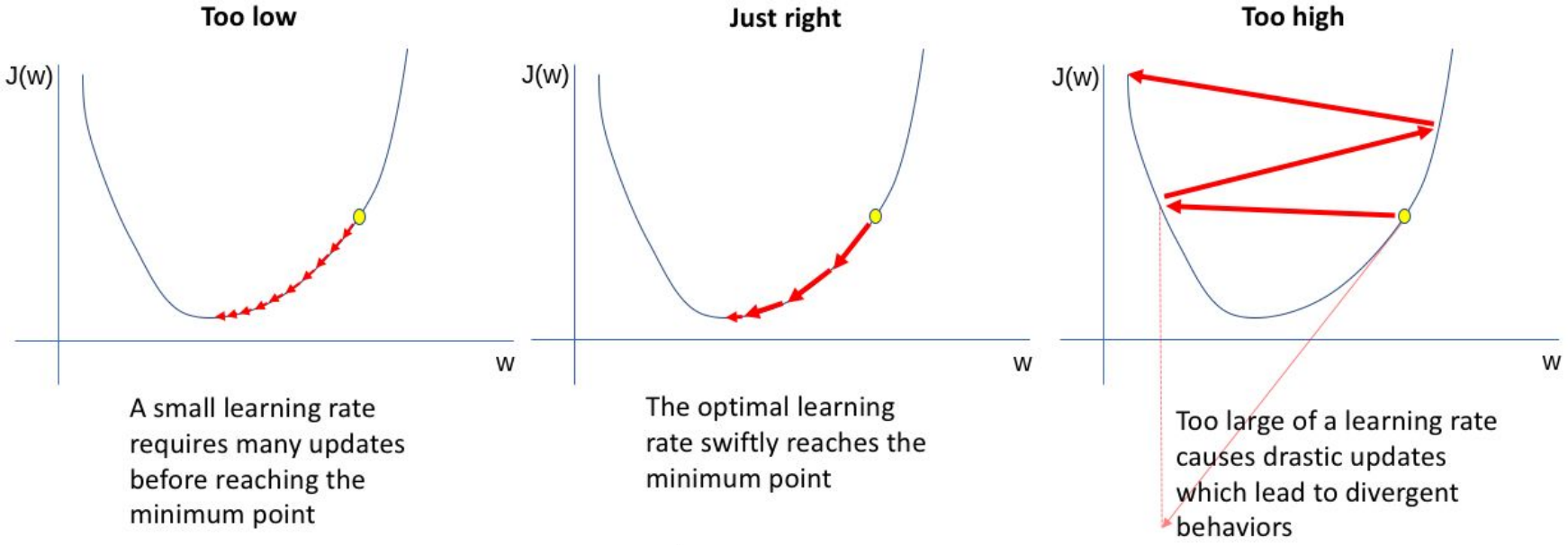
- Initialize parameters randomly
- Estimate outputs for inputs in a forward pass
- Estimate loss
- Estimate gradient of the loss function with respect to all parameters
- Move parameters in the opposite direction of gradient

$$w = w - \alpha \frac{d}{dw} J(w)$$

- Repeat these steps until loss is minimum



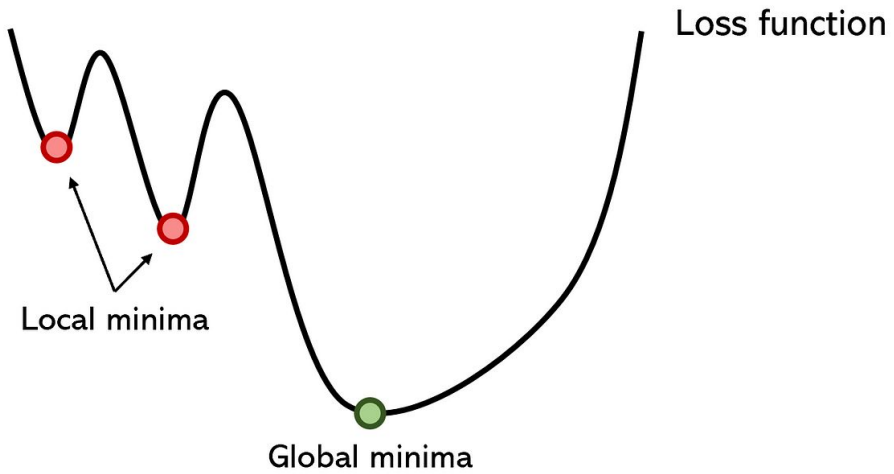
Effect of Learning Rate, α



$$w = w - \alpha \frac{d}{dw} J(w)$$

Local Vs. Global Minima

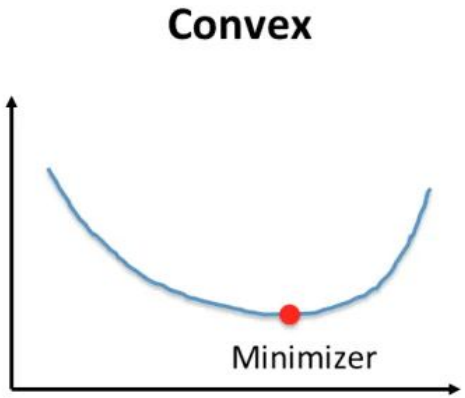
- **Local Minimum**
  A point where a function's value for a variable parameter is smaller than nearby points, but possibly larger than at a distant point.
- **Global Minimum**
  A point where a function's value for a variable parameter is smaller than at all other feasible points

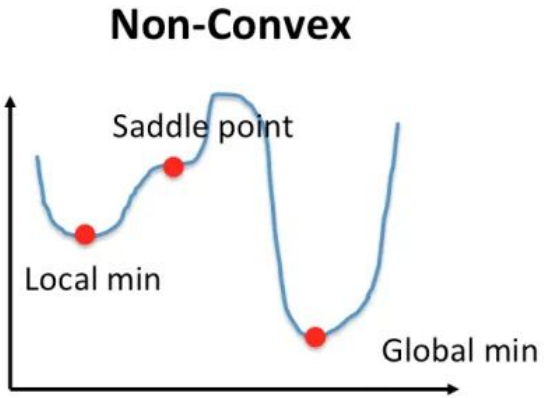


Our target is to reach the global minima, but unfortunately, most of the time,we end up at a local minima.

Convex Vs. Non-Convex Function

- Convex function has only one global minima
  - MSE
  - Hinge loss
- Non-convex function has multiple local minima





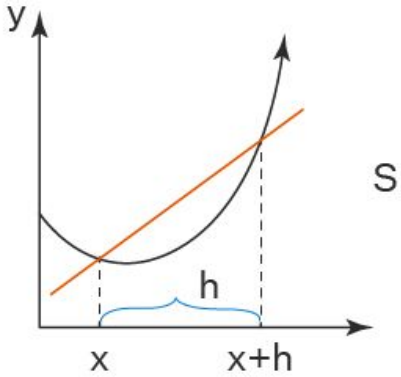
Derivative

$y = f(x)$

x: independent variable

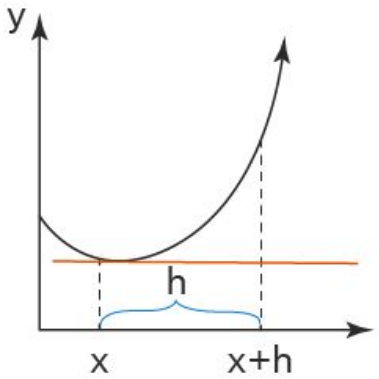
y: dependent variable

- Derivative is a way to measure the rate of change of a function with respect to an independent variable.
- It tells us how quickly a function with a single variable is changing at a specific point.



$$\text{Slope of Secant} = \frac{f(x+h) - f(x)}{h}$$

("Difference quotient")



$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

(if limit exists)

Partial Derivative

- Partial Derivative measures the rate of change of a multivariable function
- It is done with respect to one of its variables, while holding all other variables constant.

$$f(x,y,z) = 2x+3y+4z$$

$$\frac{\partial f(x,y,z)}{\partial x} = 2$$

$$\frac{\partial f(x,y,z)}{\partial y} = 3$$

$$\frac{\partial f(x,y,z)}{\partial z} = 4$$

Chain Rule

- It is a formula for finding the derivative of a composite function.

$y = (3x+3)^3$, making $t = 3x+3$

then $y = t^3$

$$\frac{dt}{dx} = 3, \qquad \frac{dy}{dt} = 3t^2$$

using the Chain Rule: $$\frac{dy}{dx} = \frac{dy}{dt}.\frac{dt}{dx}$$

$$\therefore \quad \frac{dy}{dx} = (3t^2).(3) = 9t^2$$

$$= 9(3x+3)^2 = 9(3)(x+1)(3)(x+1)$$

$$= \underline{81(x+1)^2}$$

Gradient

- The gradient is a vector of the partial derivatives of a multivariable scalar function.
- It points in the direction of the steepest increase of the function, while its negative points in the direction of the steepest decrease.

For a function of two variables, i.e., $f(x_1, x_2)$

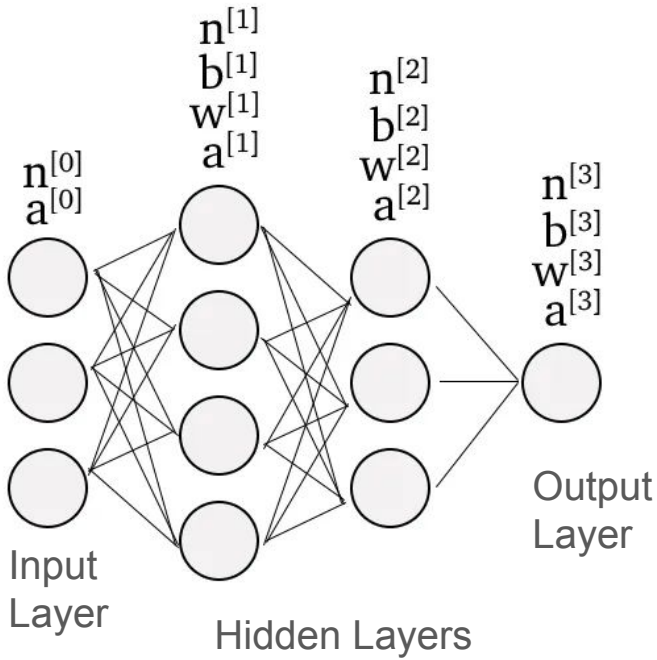
Gradient: $\nabla_x f = [\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}]$

Backpropagation Algorithm

- It is used for efficient weight adjustments to minimize the error between predicted and actual outputs.
- It works by:
  - calculating the gradient of the loss function with respect to the network's weights.
  - propagating error information backward through the network layers

A Simple Reference Neural Network

- n: number of neurons
- a: vector of activation function
- w: matrix of weights
- b: vector of biases



Formulas

- Weighted input to the neurons in layer **l**, $z^{[l]} = w^{[l]}.a^{[l-1]} + b^{[l]}$

- Activation of layer **l**, $\mathrm{a}^{[l]} = g^{[l]}(z^{[l]})$

- Say, one neuron's output **y** can be 0 or 1.

  Then binary cross-entropy cost funct $\mathrm{C} = \text{-}(\mathrm{y}\log\hat{y} + (1-y)\log(1-\hat{y}))$

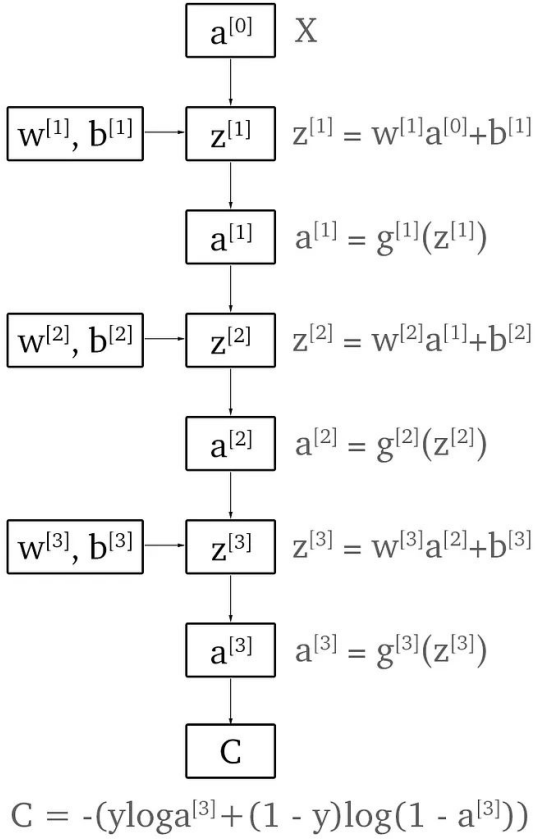
Update Rule

Alpha is the learning rate, which decides how big the updates will happen.

$$w^{[l]} = w^{[l]} - \alpha \frac{\partial C}{\partial w^{[l]}}$$

$$b^{[l]} = b^{[l]} - \alpha \frac{\partial C}{\partial b^{[l]}}$$

Forward Propagation Computation Graph



Partial Derivative of C

- Partial Derivative of C with respect to parameters of any layer, l

$$\frac{\partial C}{\partial w^{[l]}} = \frac{\partial C}{\partial z^{[l]}} \cdot \frac{\partial z^{[l]}}{\partial w^{[l]}}$$

$$\frac{\partial C}{\partial b^{[l]}} = \frac{\partial C}{\partial z^{[l]}} \cdot \frac{\partial z^{[l]}}{\partial b^{[l]}}$$

Partial Derivative of C

- Partial Derivative of C with respect to parameters of Layer-3

$$\frac{\partial C}{\partial w^{[3]}} = \frac{\partial C}{\partial a^{[3]}} \cdot \frac{\partial a^{[3]}}{\partial z^{[3]}} \cdot \frac{\partial z^{[3]}}{\partial w^{[3]}}$$

$$\frac{\partial C}{\partial b^{[3]}} = \frac{\partial C}{\partial a^{[3]}} \cdot \frac{\partial a^{[3]}}{\partial z^{[3]}} \cdot \frac{\partial z^{[3]}}{\partial b^{[3]}}$$

Partial Derivative of C

- Partial Derivative of C with respect to parameters of Layer-2

$$\frac{\partial C}{\partial w^{[2]}} = \frac{\partial C}{\partial a^{[3]}} \cdot \frac{\partial a^{[3]}}{\partial z^{[3]}} \cdot \frac{\partial z^{[3]}}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial w^{[2]}}$$

$$\frac{\partial C}{\partial b^{[2]}} = \frac{\partial C}{\partial a^{[3]}} \cdot \frac{\partial a^{[3]}}{\partial z^{[3]}} \cdot \frac{\partial z^{[3]}}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial b^{[2]}}$$

Partial Derivative of C

- Partial Derivative of C with respect to parameters of Layer-1

$$\frac{\partial C}{\partial w^{[1]}} = \frac{\partial C}{\partial a^{[3]}} \cdot \frac{\partial a^{[3]}}{\partial z^{[3]}} \cdot \frac{\partial z^{[3]}}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial a^{[1]}} \cdot \frac{\partial a^{[1]}}{\partial z^{[1]}} \cdot \frac{\partial z^{[1]}}{\partial w^{[1]}}$$

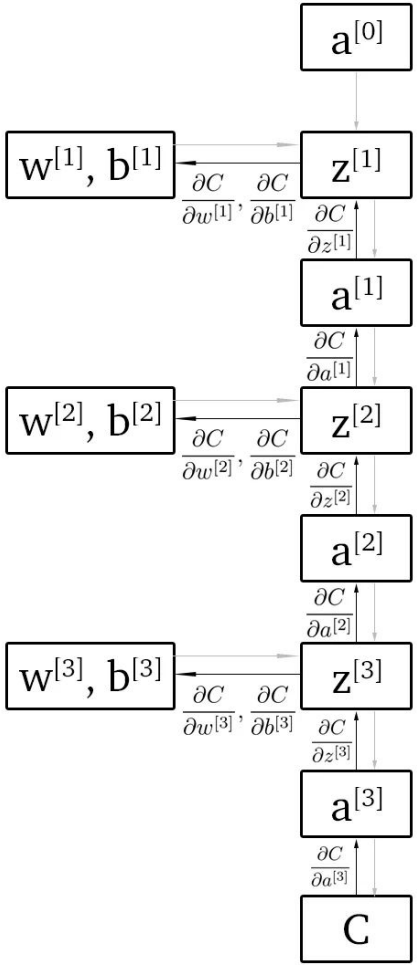
$$\frac{\partial C}{\partial b^{[1]}} = \frac{\partial C}{\partial a^{[3]}} \cdot \frac{\partial a^{[3]}}{\partial z^{[3]}} \cdot \frac{\partial z^{[3]}}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial a^{[1]}} \cdot \frac{\partial a^{[1]}}{\partial z^{[1]}} \cdot \frac{\partial z^{[1]}}{\partial b^{[1]}}$$

Backpropagation Computation Graph

- for calculating the partial derivatives of C with respect to w[*l*], b[*l*], we need to calculate:

$$\frac{\partial C}{\partial z^{[L]}}, \frac{\partial C}{\partial z^{[l]}}, \frac{\partial z^{[l]}}{\partial w^{[l]}}, \frac{\partial z^{[l]}}{\partial b^{[l]}}$$

- error information is propagated backward through the network layers for updating the weights



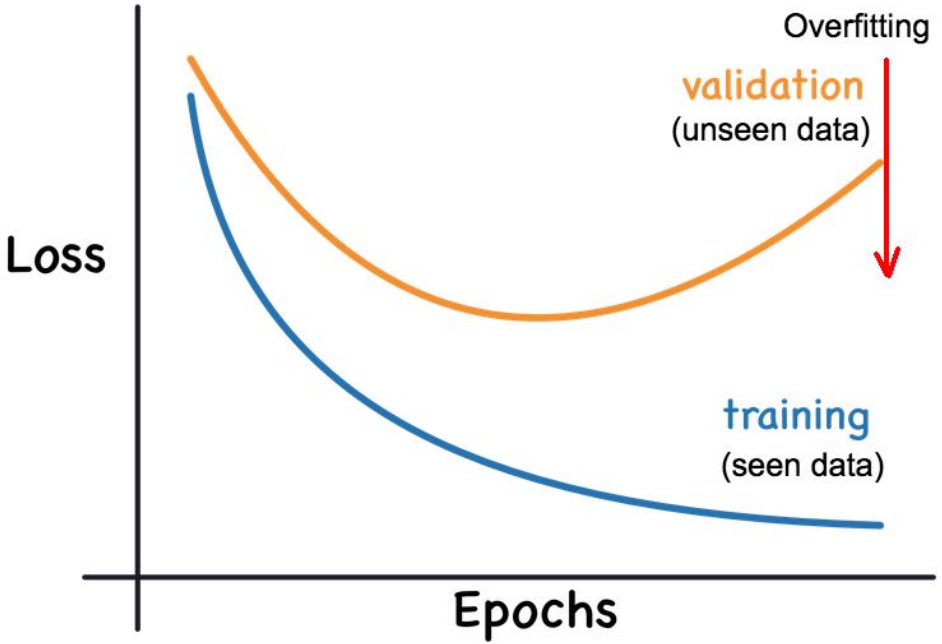
Overfitting and Underfitting

Overfitting

Overfitting occurs when:

- the model is so closely aligned to the training data that it does not know how to respond to new data.
- the model memorizes training data

Memorizing is not learning



Reasons behind Overfitting

Overfitting can happen because:

- Model is too complex; it memorizes very subtle patterns in the training data that don’t generalize well.
    - Model has too many parameters
- The training data size is too small for the model complexity and/or contains large amounts of irrelevant information.
- During training model sees a small dataset for too many times i.e., too many epochs were used during training.

How to Avoid Overfitting

- **Data Augmentation**
  - When data availability is limited, data augmentation is a method to artificially expand the data points of the training data set by adding randomized versions of the existing data to the data set.
- **Synthetic Data**
  - Synthetic data generated by generative model can bring variability in available data, increase size of training dataset and reduce overfitting issue to some extent.
- **Data Clean Up**
  - Data noisiness contributes to overfitting. One common approach to reduce undesired data points is to remove outliers from the data.

How to Avoid Overfitting

- **Model Validation**
  - The error of an overfitted model is low when computed for the training data. We need to test performance of our model on a separate data set (i.e., validation data set) before using real-time application data.
- **Regularization**
  - Regularization applies either a penalty for complexity or roughness to the model. By introducing additional information into the model, regularization algorithms can deal with multicollinearity and redundant predictors by making the model more parsimonious and accurate.
  - https://www.youtube.com/watch?v=wNdYpD2YhSk&t=331s

Regularization

- **Regularization** is a technique used in machine learning to **prevent overfitting** by discouraging overly complex models.
- It adds a constraint or penalty to the model so that it learns **simpler and more generalizable patterns**.

$$\mathcal{L}(w) = \text{Loss} + \lambda \cdot \text{Penalty}$$

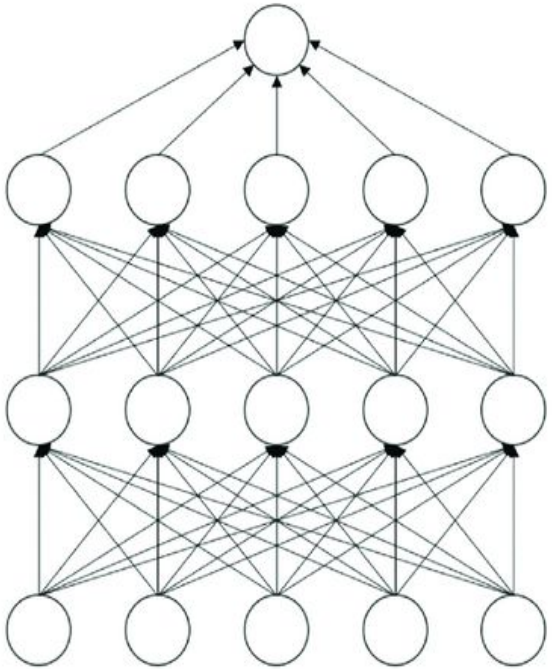
- $\mathcal{L}(w)$: objective function
- λ: regularization strength
- Penalty: controls model complexity

Effect of Regularization

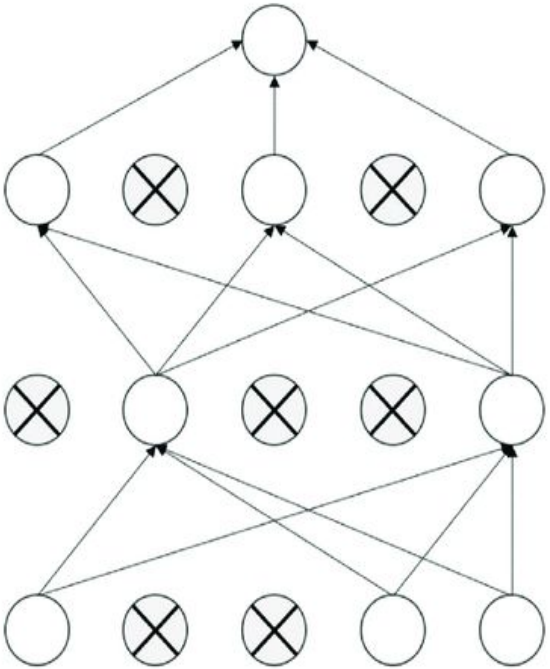
- Visual Idea:
  - Without regularization → very wiggly curve
  - With regularization → smoother curve
- Regularization sacrifices a bit of training accuracy to gain better real-world performance.
- Regularization **can hurt performance** if:
  - λ is too large
  - Model becomes too simple
  - Important patterns are suppressed

Dropout

- **Dropout** is a regularization technique used to **reduce overfitting**.
- In dropout, individual nodes are excluded in various training runs.



Standard Neural Net



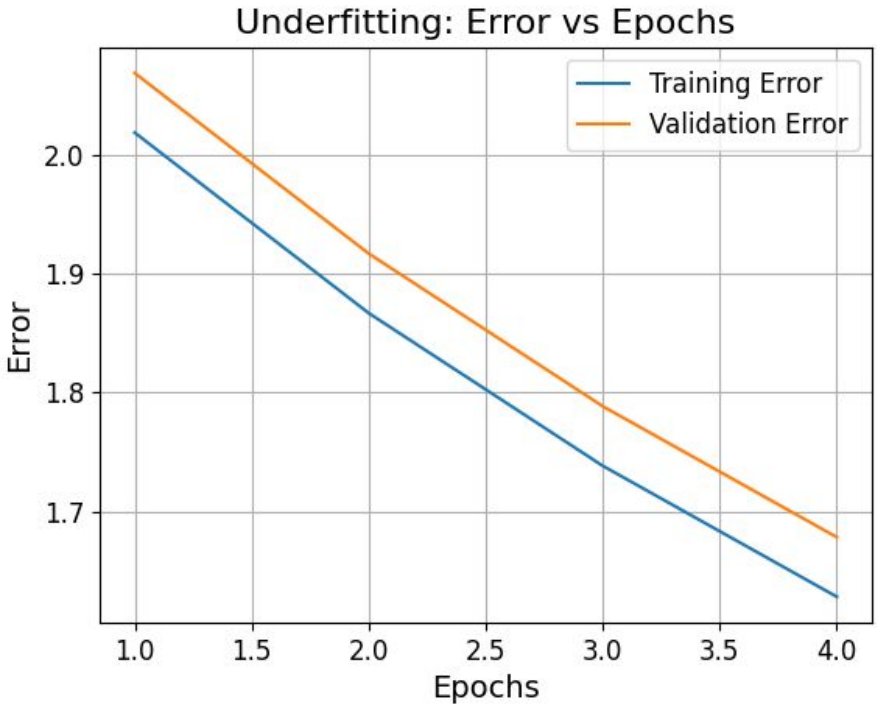
After applying dropout

Dropout

- It activates during training phase only, not in validation and testing phase
- Forward Pass:
  - A certain percentage of neurons in the dropout layer are randomly picked to be "dropped" (deactivated).
  - The output of the dropped neurons is set to zero, effectively removing them from the network's calculations for that specific iteration.
- Backward Pass:
  - The gradients for the dropped neurons are effectively ignored. They don't contribute to the weight updates.
  - The gradients for the active neurons are scaled by a factor that accounts for the dropout probability

Underfitting

- Underfitting is the opposite concept of overfitting
- Underfitting happens when the model is too simple to learn the data, leading to poor performance on both training and test sets.
- When the prediction error on both the training and validation dataset is high, and the difference between them is very minimal, the model is said to have under fitted.



Causes of Undefitting

1. Model Too Simple
   - Using a model with low capacity which cannot capture complex relationships
2. Too Much Regularization
   - Large regularization parameter (λ) forcing weights to be very small
3. Insufficient Training (Too Few Epochs)
   - Model hasn’t learned enough yet
4. Poor Feature Representation
   - Important features are missing
   - Features are not informative

Causes of Undefitting

5. Improper Hyperparameters
   - Learning rate too high or too low
   - Too few neurons/layers
6. Overly Strong Constraints
   - Heavy dropout
   - Excessive pruning
7. Noisy or Incorrect Data Handling
   - Wrong labels and Poor preprocessing

How to Fix Underfitting

- Increase model complexity
  - More layers / neurons
- Reduce regularization
  - Lower λ
- Train longer
  - More epochs
- Better features
  - Feature engineering / better representation

Overfitting vs Underfitting

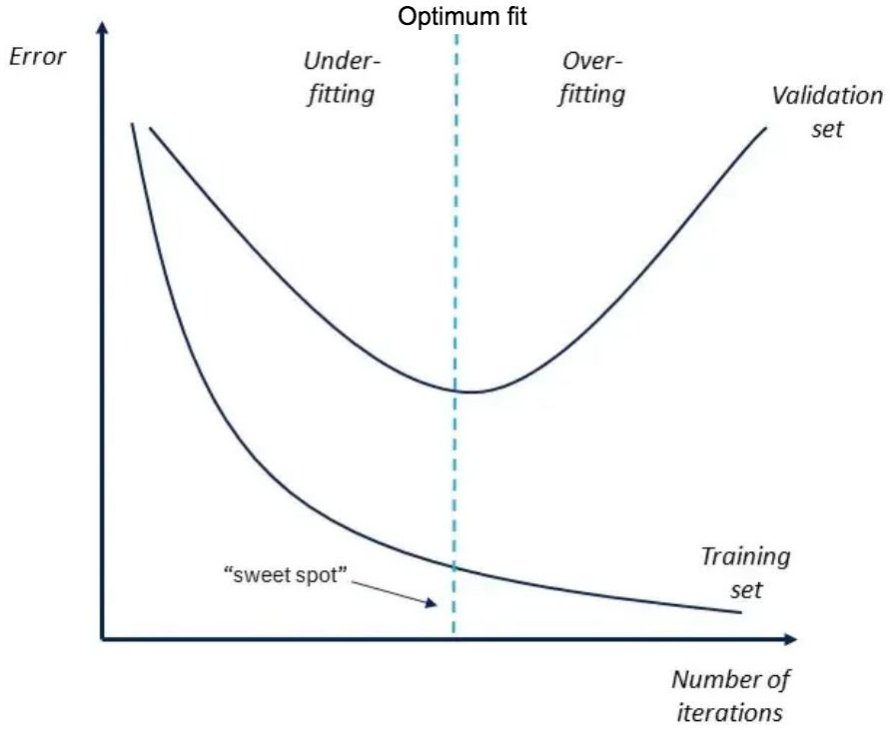
When only looking at the computed error of a model for the training data, overfitting is harder to detect than underfitting.

Error	Overfitting	Right Fit	Underfitting
Training	Low	Low	High
Test	High	Low	High

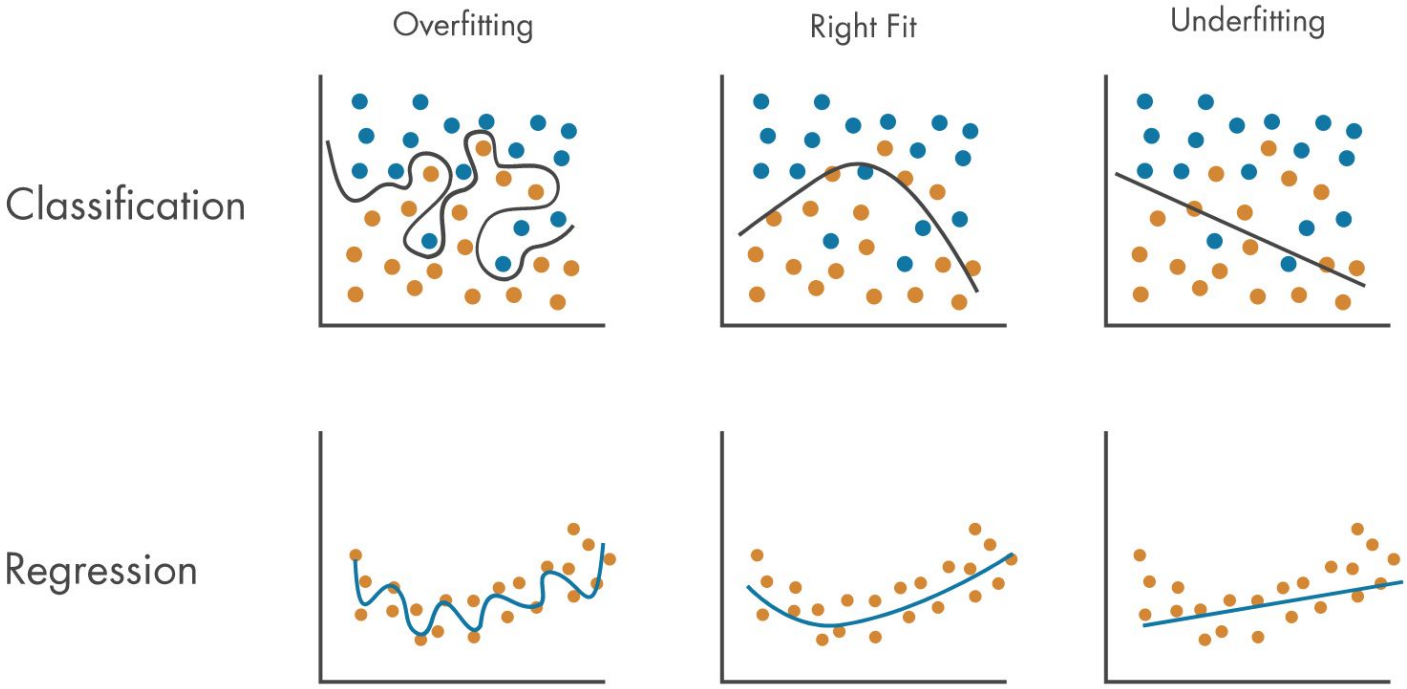
More details: https://www.mathworks.com/discovery/overfitting.html

Optimal fit

- **Optimal fit** is the situation where a model **learns the true pattern of the data** without being too simple (underfitting) or too complex (overfitting).
- An optimal fit occurs when a model achieves the **best balance between training performance and generalization to new data**.

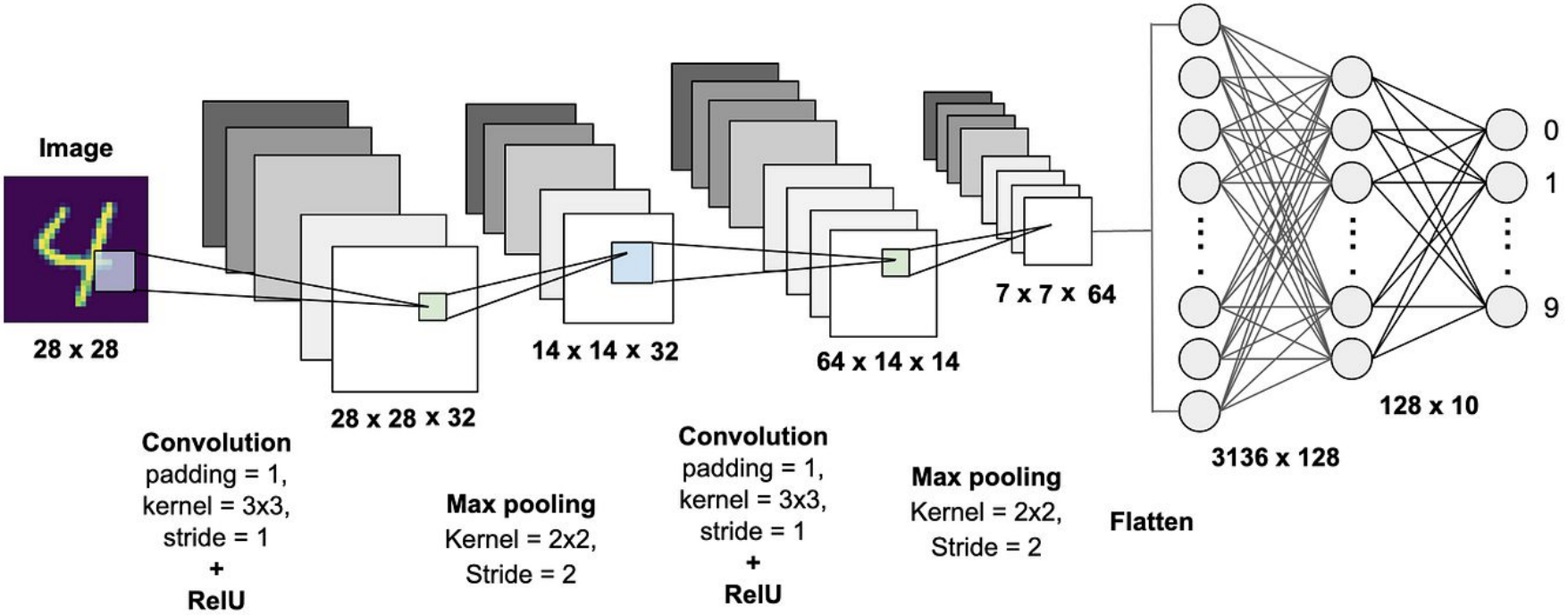


Overfitting - Underfitting in Classification and Regression



CNN

CNN Classifier



Convolution

- Convolution is a mathematical operation where a **filter (kernel)** slides over the input and computes weighted sums to produce a feature map.

$$y(i,j) = \sum_{m,n} x(i+m, j+n) \cdot w(m,n)$$

- $x \rightarrow$ input (image)
- $w \rightarrow$ filter/kernel
- $y \rightarrow$ output (feature map)

Input image



Convolution Kernel

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Feature map



Kernel

- In computer vision, kernel is a (usually) small matrix of numbers that is used in image convolutions.
- Different sized kernels containing different patterns of numbers produce different results under convolution.
- Generally, small odd numbers such as 3, 5, or 7 are used as the size of a kernel.
- Popular human-decided kernels:
  - Gaussian Blur Kernel
  - Laplacian Kernel
  - Prewitt Kernel
  - Sobel Kernel

Sobel Kernel

$K_{Vertical} =$

-1	0	1
-2	0	2
-1	0	1

$K_{Horizontal} =$

1	2	1
0	0	0
-1	-2	-1

$K_{Left\ Diagonal} =$

0	1	2
-1	0	1
-2	-1	0

$K_{Right\ Diagonal} =$

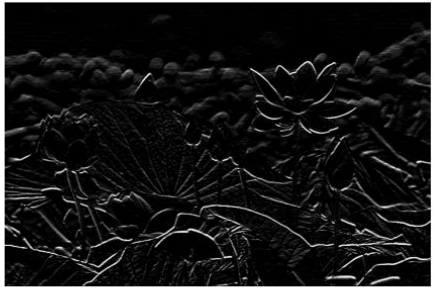
-2	-1	0
-1	0	1
0	1	2

Effect of Well-Known Sobel Kernel

Grayscale



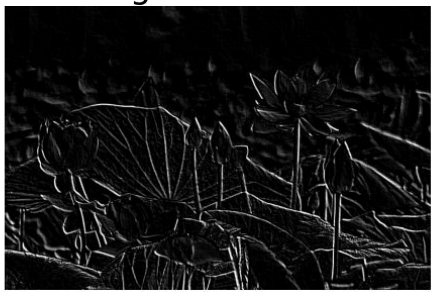
Horizontal Convolution



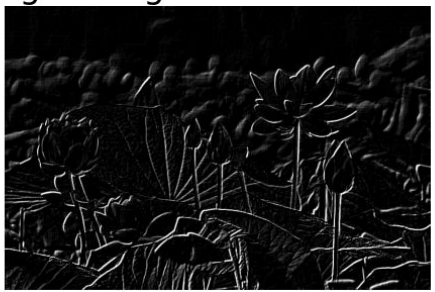
Vertical Convolution



Left-Diagonal Convolution



Right-Diagonal Convolution

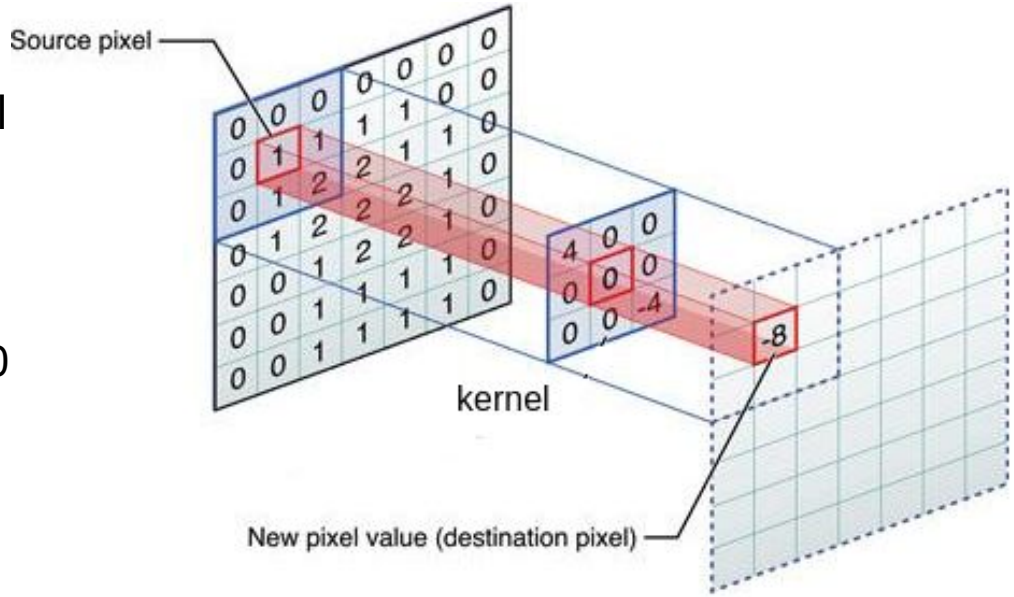


Convolution for Single-Channel Input & Single-Channel Output

Convolution layer adds each element of a feature map to its local neighbors, weighted by the kernel.

Source pixel: 1

Destination pixel: 4x0 + 0x0 + 0x0 + 0x0 + 1x0 + 2x0 + 0x0 + 1x0 + (-4)x2 = -8

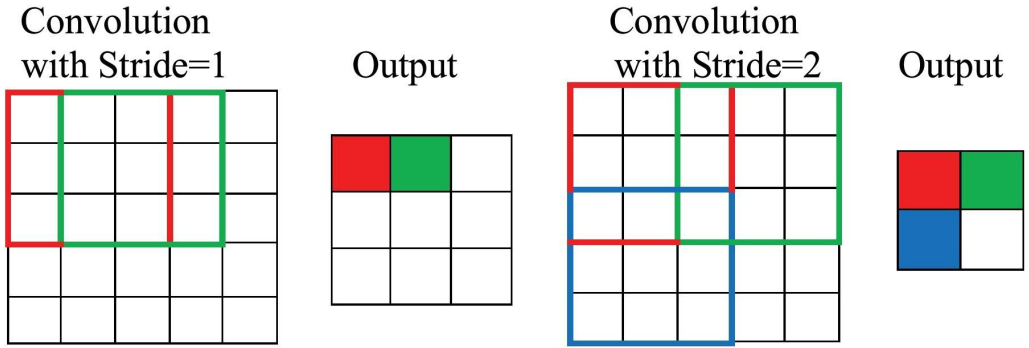


Total number of multiplication for one pixel = kernel_height x kernel_width

Strides

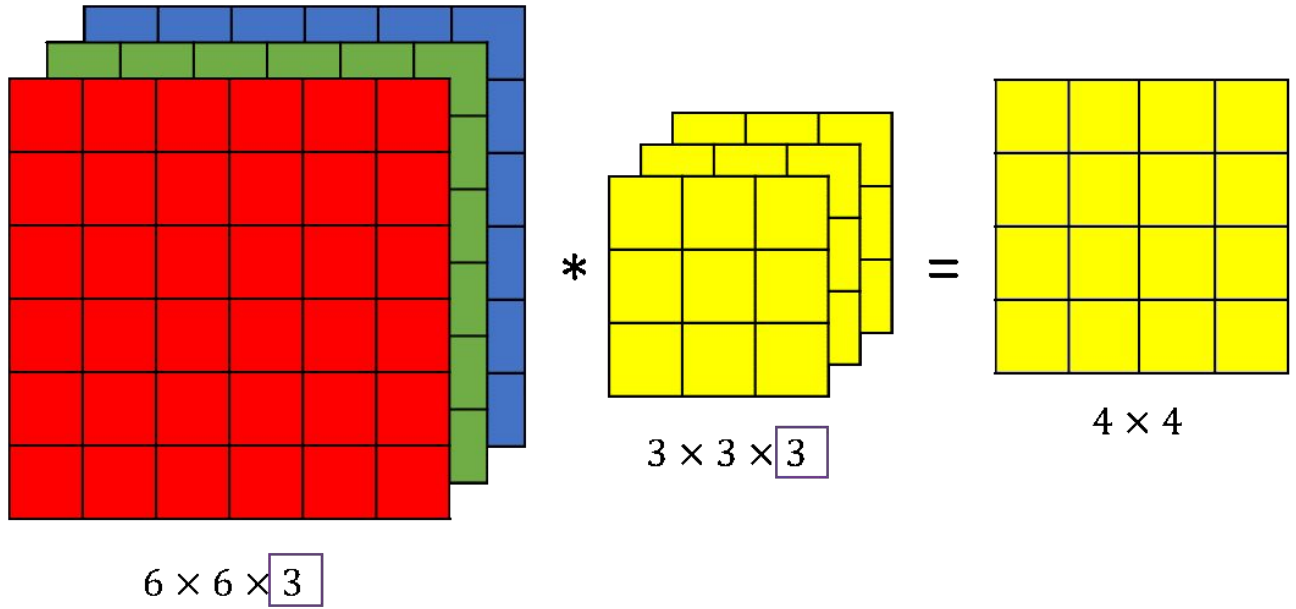
Stride decides how many pixels the filter matrix moves across the input matrix.

Larger stride value means more reduced output matrix.



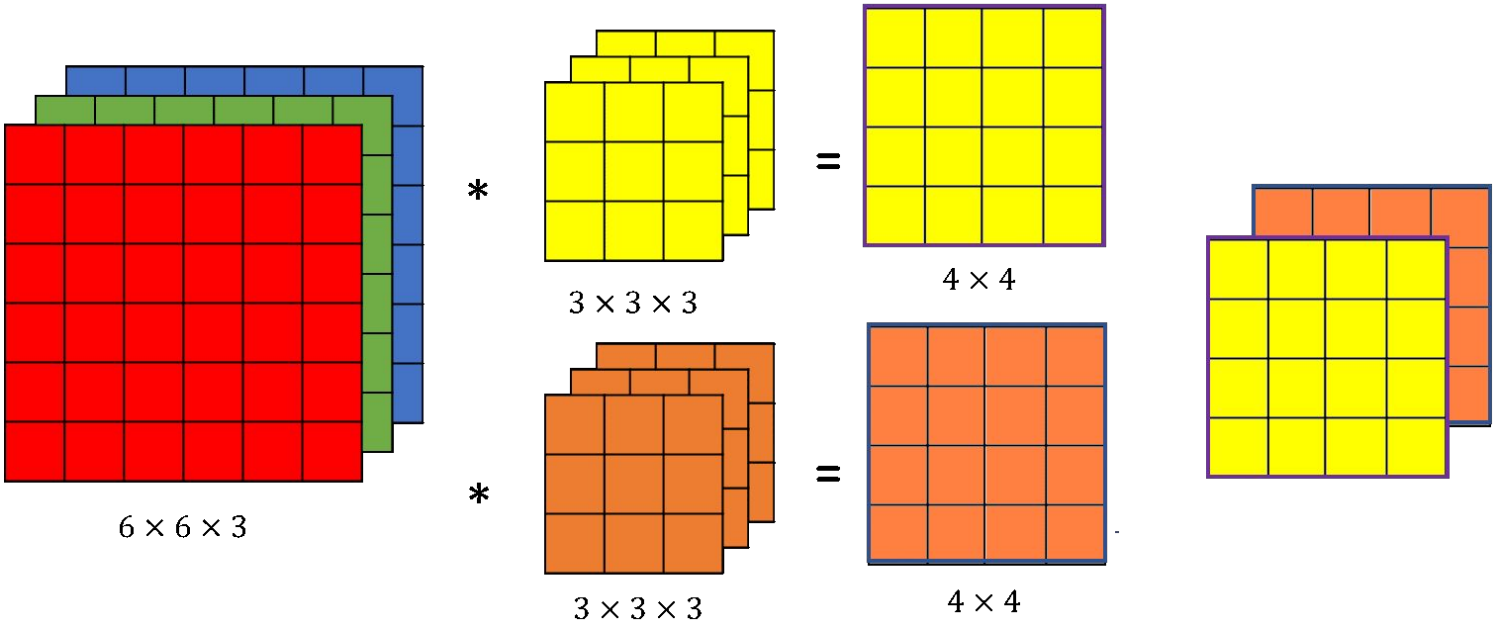
Convolution for Multi-Channels Input & Single-Channel Output

- Number of Kernel's Channels = Number of Input Feature Map's Channels



Convolution for Multi-Channels Input & Multi-Channels Output

- Number of Kernels = Number of Output Feature Map's Channels



Feature Map

A feature map:

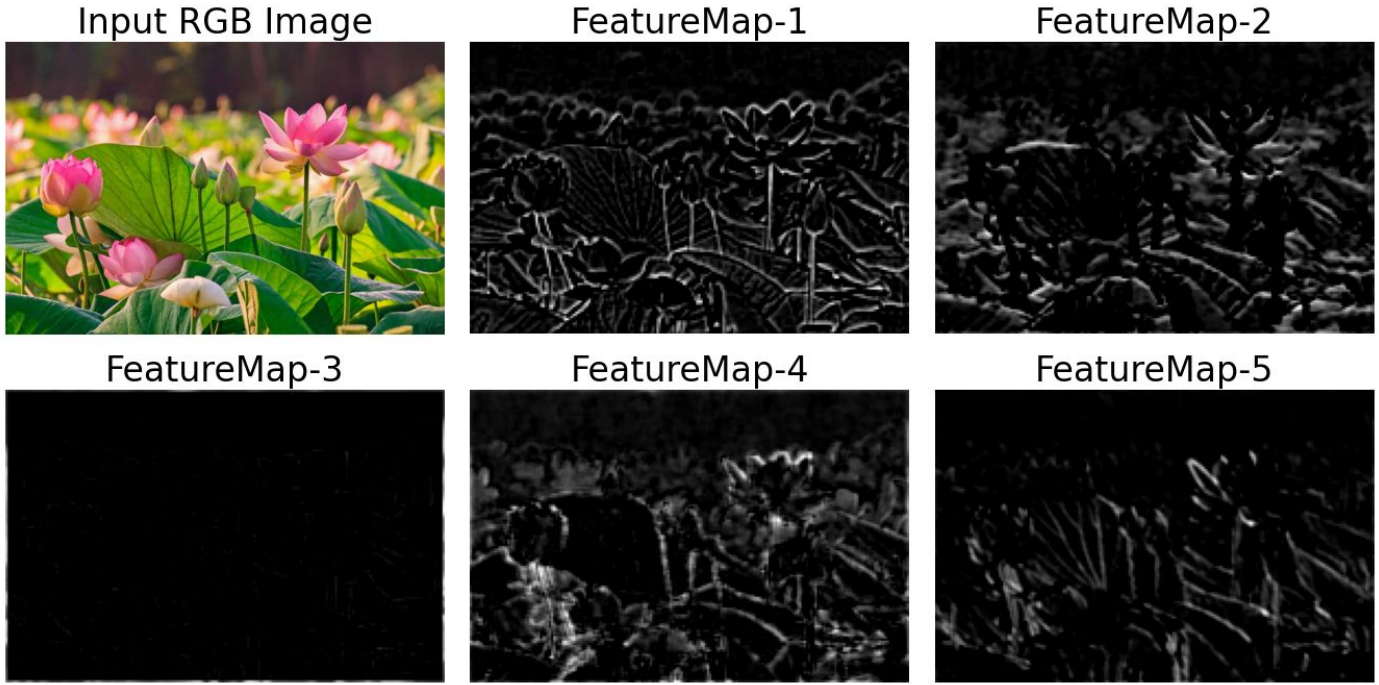
- is a 2D matrix in a hidden layer
- represents the output of a convolutional layer
- captures specific features or patterns in the input data.

In a CNN:

- Instead of human-decided kernels/filters, various automatically learned kernels/filters are used to generate different feature maps

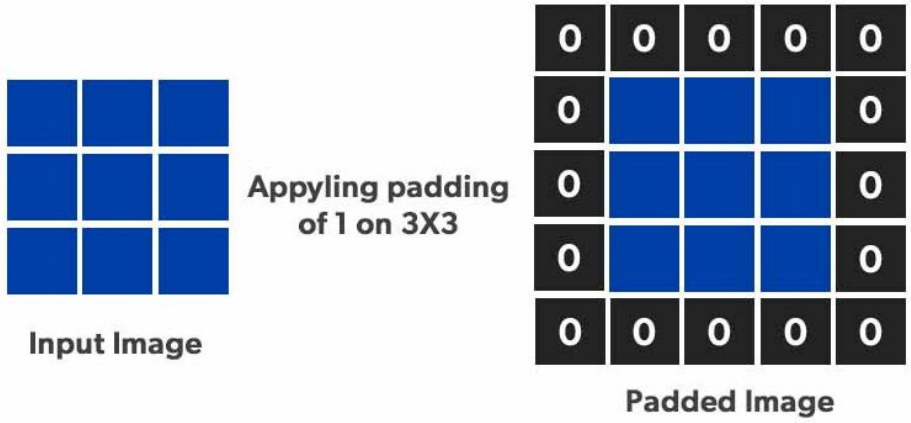
Feature Maps of Pre-trained VGG16

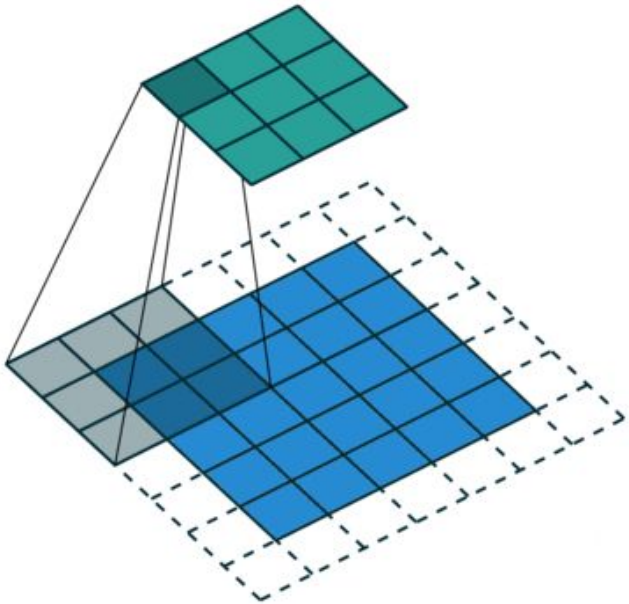
- Layer: block3_conv1



Padding

Padding preserves the spatial size of the input so that the output after convolution remains the same size or desired size.





Downsampling

Downsampling means **reducing the spatial size** (height/width) of data.

Purpose

- Reduce computation
- Extract high-level (abstract) features
- Increase receptive field

Common Methods

- **Pooling** (Max Pooling, Average Pooling, Global Average Pooling)
- Strided convolution (stride > 1)
- Patch merging (in hierarchical models like Swin)

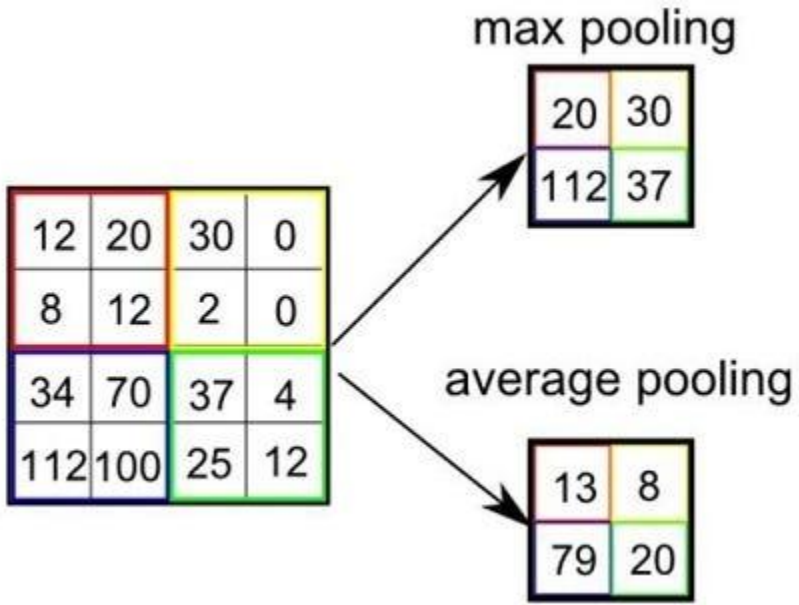
Pooling Layer

It reduces the spatial dimensions of feature maps, thereby:

- decreases computational cost
- makes the network more robust to variations in the input

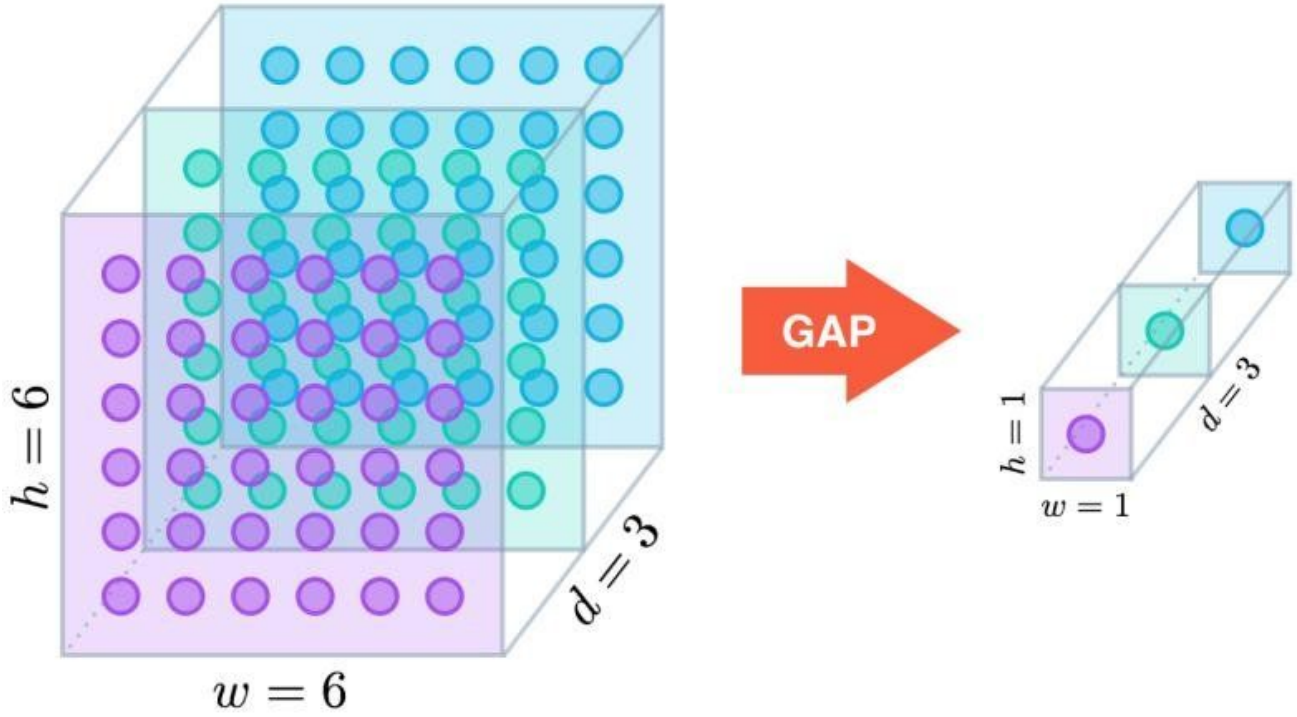
Different Types of Pooling Layers:

- Max Pooling
- Average Pooling
- Global Pooling
- Stochastic Pooling (not very common)

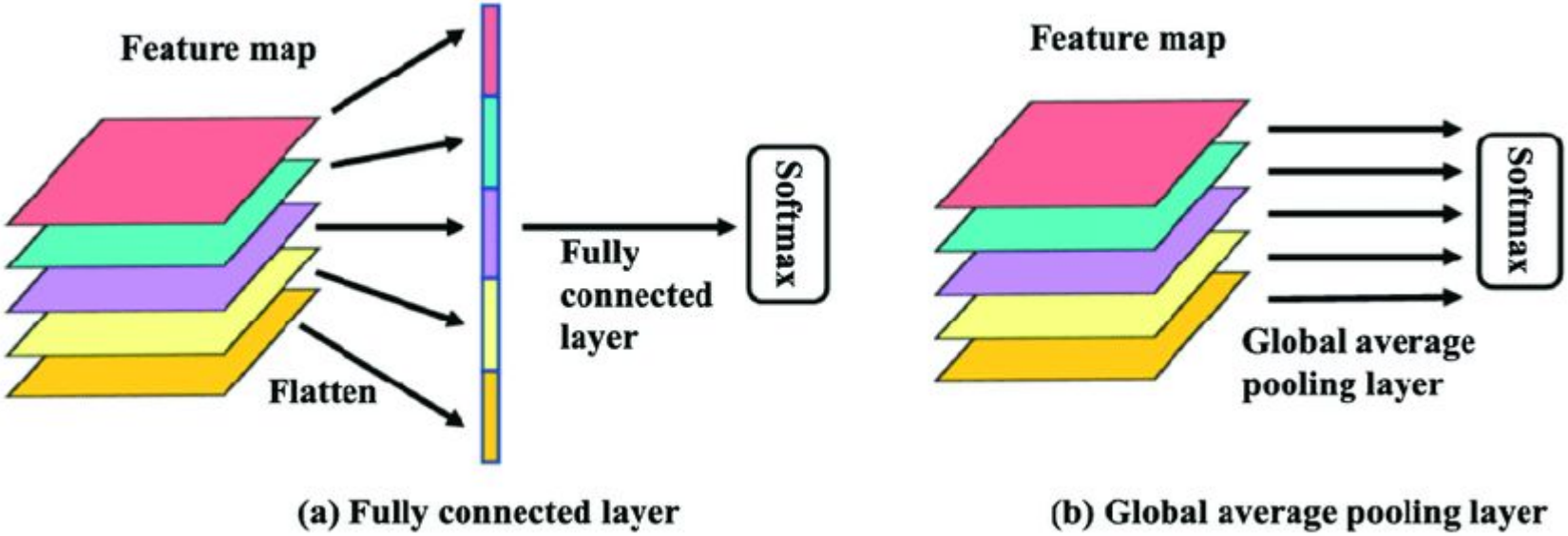


Global Average Pooling (GAP)

GAP reduces each feature map to a single value by averaging all of its elements.

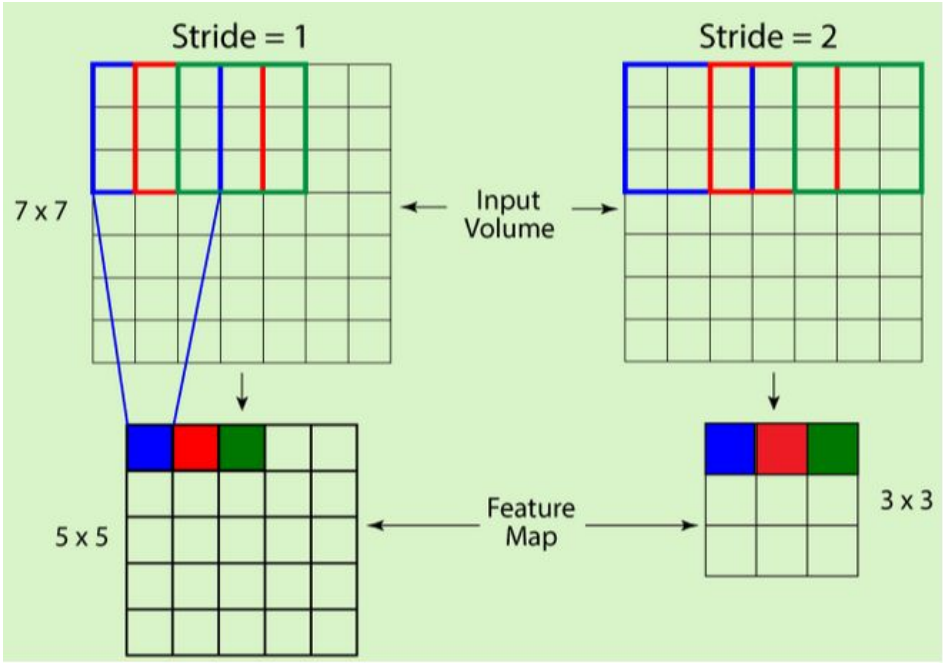


GAP


Feature map
Flatten
Fully connected layer
Softmax
(a) Fully connected layer
Feature map
Global average pooling layer
Softmax
(b) Global average pooling layer

Downsampling by Convolutional Layer

- Without padding, setting appropriate stride value, we can downsample feature map by convolution.
- Convolution layer with larger strides and no padding
  - It is a learnable operation, whereas pooling is a fixed operation
  - Some research works showed better performance



Transfer Learning

Transfer Learning

- **Transfer learning** is a technique where a model trained on one task is **reused for a different but related task**.
- It leverages knowledge learned from a pre-trained model to on a new task,.
  - improve performance especially when data is limited
  - reduce training time
  - use powerful pre-trained models
  - reduce data amount

Steps of Transfer Learning

- Step 1: Choose a Pre-trained Model
  - Select a model trained on a large dataset
  - Examples:
    - CNN (ResNet, VGG)
    - Transformer models

- Step 2: Load the Model
  - Import pre-trained weights
  - Remove or modify the final layer

- Step 3: Freeze Base Layers
  - Keep early layers unchanged since they layers capture general features

Steps of Transfer Learning

- Step 4: Modify Output Layer
  - Replace final layer according to your task
  - Example:
    - Old: 1000 classes
    - New: 2 classes
- Step 5: Train New Layers
  - Train only the modified layers
  - Use only new dataset
- Step 6: Fine-Tuning (Optional)
  - Unfreeze some layers
  - Train with a low learning rate

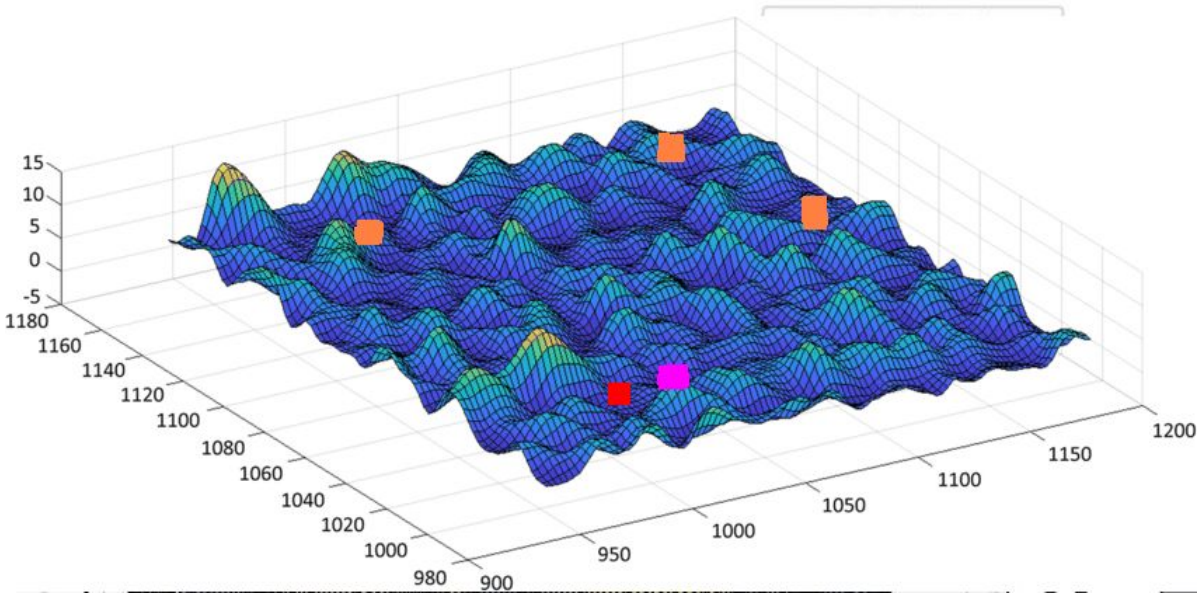
Advantage of Transfer Learning

Training from scratch:

- random weights → need lots of data to learn everything

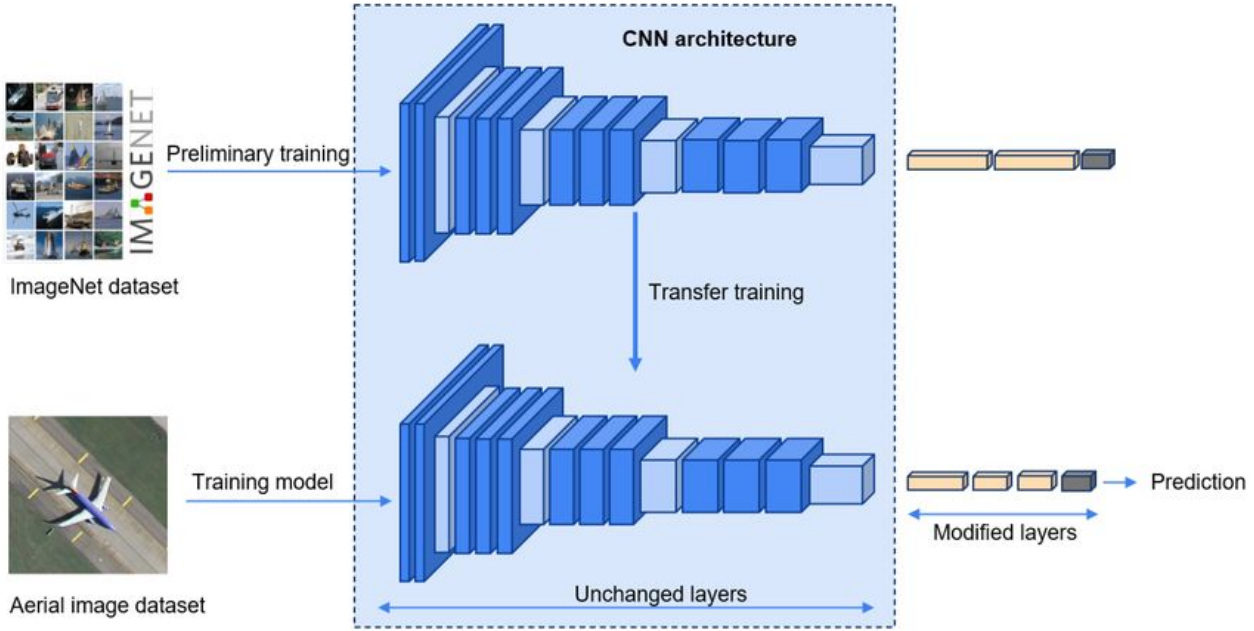
Transfer learning:

- **pre-trained weights ≈ good starting point**



Transfer Learning by Pre-trained Model

- Unchanged pre-trained model can be used as a feature-extractor.
- Fine-tuned pre-trained model is a better feature-extractor.

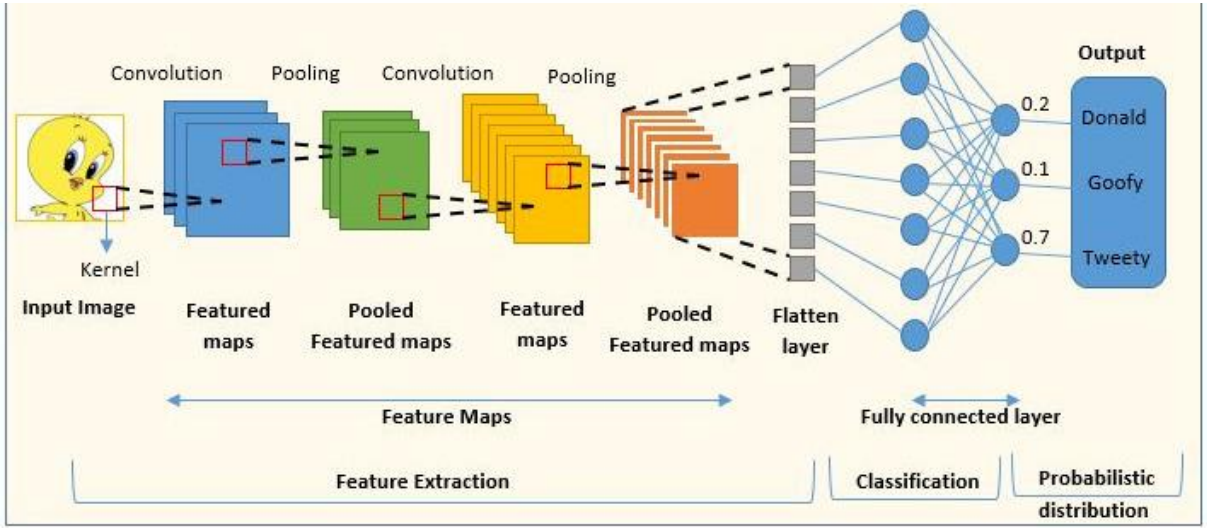


Pre-trained Classifiers

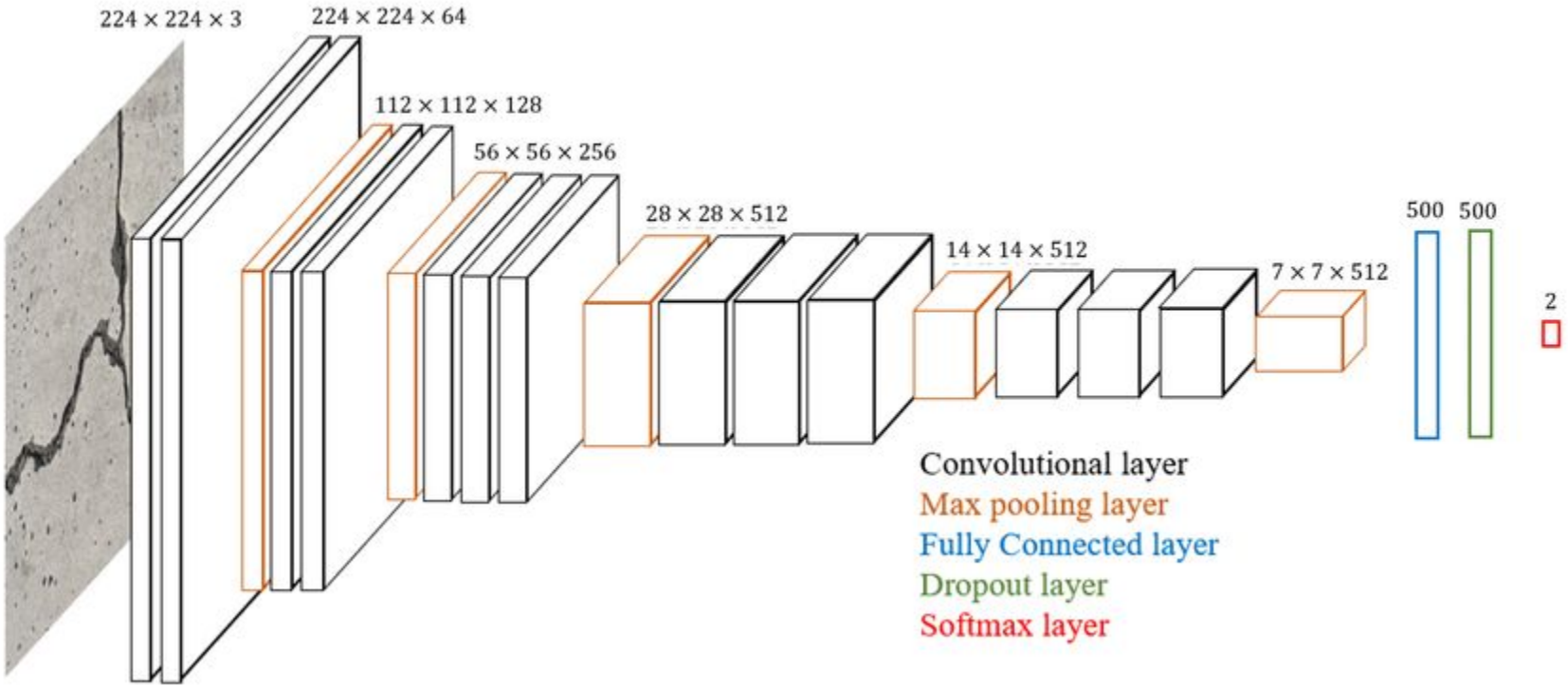
- Pre-trained classifiers consist of a series of convolutional layers and downsampling layers.
- For classification, fully connected layers are added after convolutional layers.

Popular CNNs:

- VGG
- InceptionNet
- ResNet
- EfficientNet
- MobileNet
- XceptionNet
- DenseNet



VGG16


224 × 224 × 3
224 × 224 × 64
112 × 112 × 128
56 × 56 × 256
28 × 28 × 512
14 × 14 × 512
7 × 7 × 512
500
500
2
Convolutional layer
Max pooling layer
Fully Connected layer
Dropout layer
Softmax layer

Pre-trained Classifiers Provided by Keras API Team

- Models were trained by ImageNet 1K dataset for 1000 classes

https://keras.io/api/applications/

- Most of the time, we depend on these pre-trained models

Model	Size (MB)	Top-1 Accuracy	Top-5 Accuracy	Parameters	Depth	Time (ms) per inference step (CPU)	Time (ms) per inference step (GPU)
Xception	88	79.0%	94.5%	22.9M	81	109.4	8.1
VGG16	528	71.3%	90.1%	138.4M	16	69.5	4.2
VGG19	549	71.3%	90.0%	143.7M	19	84.8	4.4
ResNet50	98	74.9%	92.1%	25.6M	107	58.2	4.6
ResNet50V2	98	76.0%	93.0%	25.6M	103	45.6	4.4
ResNet101	171	76.4%	92.8%	44.7M	209	89.6	5.2
ResNet101V2	171	77.2%	93.8%	44.7M	205	72.7	5.4
ResNet152	232	76.6%	93.1%	60.4M	311	127.4	6.5
ResNet152V2	232	78.0%	94.2%	60.4M	307	107.5	6.6
InceptionV3	92	77.9%	93.7%	23.9M	189	42.2	6.9
InceptionResNetV2	215	80.3%	95.3%	55.9M	449	130.2	10.0
MobileNet	16	70.4%	89.5%	4.3M	55	22.6	3.4
MobileNetV2	14	71.3%	90.1%	3.5M	105	25.9	3.8
DenseNet121	33	75.0%	92.3%	8.1M	242	77.1	5.4
DenseNet169	57	76.2%	93.2%	14.3M	338	96.4	6.3

Basic Code

Code for Loading Pre-trained Model

```
#--- Load the pre-trained model, VGG16, with head
from tensorflow.keras.applications import vgg16
vgg16_model = vgg16.VGG16()

#--- Display architecture
vgg16_model.summary(show_trainable = True)

#--- Load the pre-trained model, VGG16, without head
vgg16_model = vgg16.VGG16(include_top = False)
vgg16_model.summary(show_trainable = True)
```

Code for Loading MNIST Fashion Dataset

```
#-- Load data
from tensorflow.keras.datasets import fashion_mnist
(trainX, trainY), (testX, testY) = fashion_mnist.load_data()
print(trainX.shape, trainX.dtype, trainY.shape, trainY.dtype)

#--- Cross check
plt.imshow(trainX[0])
plt.title(trainY[0])
plt.show()
plt.close()
```

Code for Training, Predicting and Evaluating

```
#--- Train a model
model.compile(loss_function, metric_list)
model.fit(trainX, trainY, epochs, batch_size, validation)

#--- Predict by a model
predictY = np.argmax(model.predict(testY))

#--- Evaluate by a model
model.compile(loss_function, metric_list)
loss, metric1, metric2, … = model.evaluate(testX, testY)
```

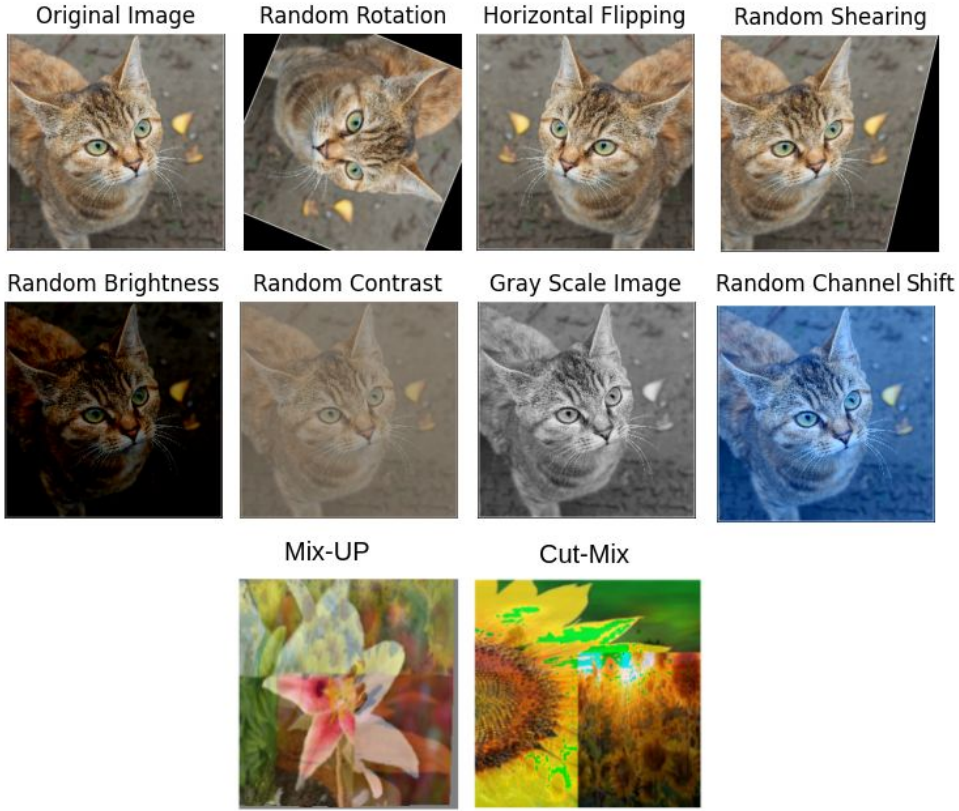
Data Augmentation

Data Augmentation (DA)

When training data is limited and we need to train a complicated NN with huge number of parameters, DA can save us by:

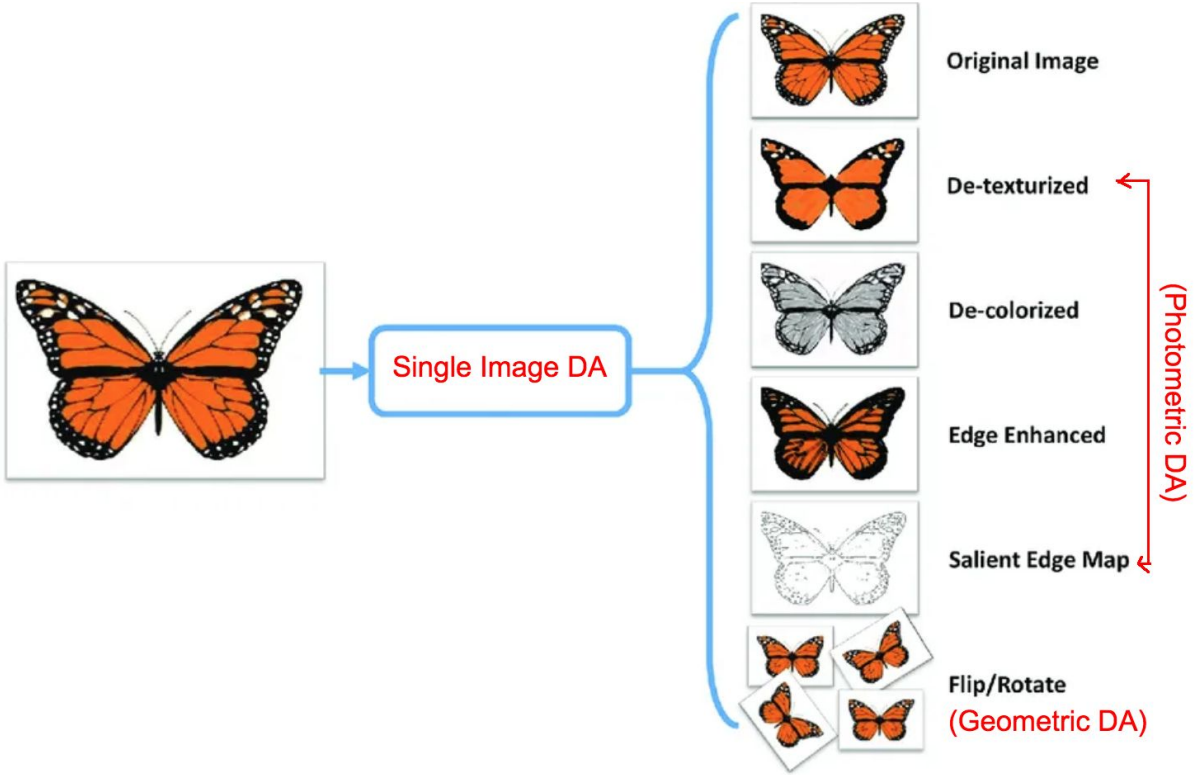
- Increasing amount of data
- Increasing variability in data
- Reducing possibility of overfitting

Random DA should not be used for validation and test sets during validation and testing



Types of DA

1. DA by a single image
   a. Photometric DA
   b. Geometric DA
2. DA by multiple images
   a. MixUp
   b. CutMix



Tensorflow’s Support for DA

- ImageDataGEnerator: tf.keras.preprocessing.image.ImageDataGenerator | TensorFlow v2.16.1
- Building powerful model with Little Data: Building powerful image classification models using very little data

```
datagen = ImageDataGenerat
        rotation_range=40,
        width_shift_range=0.2,
        height_shift_range=0.2,
        shear_range=0.2,
        zoom_range=0.2,
        horizontal_flip=True,
        fill_mode='nearest')
```



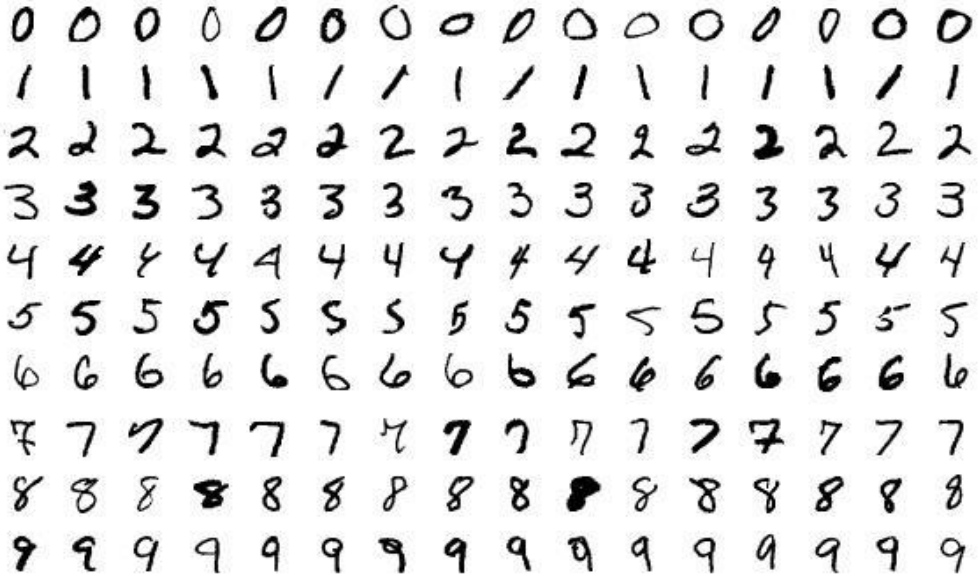
Data Generator

- In by-default situation, we need to load whole dataset into memory which cause OOM (Out-Of-Memory) problem for a large dataset or a device with a smaller memory.
- We need to complete all preprocessing tasks and save preprocessed data in the hard disks.
  - It demands extra storage space specially when we need to figure out appropriate preprocessing steps.
- Data generator is used to:
  - generate customized batch.
  - Handle large dataset while avoiding OOM (Out of Memory) problem
  - Pre-process data on run-time
- Code help: https://stanford.edu/~shervine/blog/keras-how-to-generate-data-on-the-fly

Datasets

MNIST Digit Dataset for Classification

- 70000 images 28 x 28
  - 60000 for training
  - 10000 for testing
- 10 classes
  - 0
  - 1
  - 2
  - 3
  - 4
  - 5
  - 6
  - 7
  - 8
  - 9



MNIST Fashion Dataset for Classification

- 70000 images 28 x 28
  - 60000 for training
  - 10000 for testing
- 10 classes
  - 0: T-shirt/top
  - 1: Trouser
  - 2: Pullover
  - 3: Dress
  - 4: Coat
  - 5: Sandal
  - 6: Shirt
  - 7: Sneaker
  - 8: Bag
  - 9: Ankle boot



Other Datasets for Classification

- CIFAR 10 dataset
  - https://www.cs.toronto.edu/~kriz/cifar.html
  - 60000, 32x32 colour images in 10 classes, with 6000 images per class
  - 50000 training images and 10000 test images
- CIFAR 100 dataset
  - https://www.cs.toronto.edu/~kriz/cifar.html
  - 100 classes containing 600 images each
  - 500 training images and 100 testing images per class
- ImageNet 1000 dataset
  - https://www.image-net.org/
  - 1000 object classes
  - 1281167 training images, 50000 validation images and 100000 test images.